
Detecting Simplification in Neural Network Dynamics from Scalar Logs: Active Dimension in Controlled Recurrent Systems

Anonymous Author
Anonymous Institution

Abstract

Neural network dynamics can simplify during learning, but measuring this change often requires access to high-dimensional states or costly diagnostics. We develop a method that detects simplification from a single scalar log by combining delay reconstruction with the pooled MacKay–Ghahramani (MG) dimension estimator. Delay-embedding theory motivates interpreting the reconstructed geometry in terms of active components. Synthetic experiments test this interpretation, including the distinction between independent phases and harmonics. In learned systems, MG tracks restricted parameter motion and latent information loss. It also supports model and policy selection; in self-supervised learning, a gradient-norm log ranks configurations better than several embedding-based proxies in the tested setting. Once the log is available, analysis requires no model access. A sampled-query implementation further reduces computation while preserving forecast choices in our benchmark. Exact dimensional interpretation requires suitable recurrence and observation conditions. For general training logs, we evaluate MG as a change statistic against independent measurements and competing scalar methods. This yields a low-cost geometric diagnostic with demonstrated uses across distinct learning tasks.

1 INTRODUCTION

A training loss can decrease while the learned representation loses useful information. A recurrent network

can reproduce its target more accurately while its activity settles onto a simpler cycle. Both changes matter, but measuring them directly may require saving activations, analyzing parameter trajectories, or propagating tangent dynamics. We ask how much of this information can be obtained from the scalar logs already available during training.

A scalar value cannot describe a network state, but its history records how that state evolves. Delay-embedding theory explains when delayed observations reconstruct the dynamics (Takens, 1981; Sauer et al., 1991; Stark, 1999; Stark et al., 2003). We apply pooled MacKay–Ghahramani (MG) estimation (MacKay and Ghahramani, 2005; Levina and Bickel, 2004) to this reconstruction. Under the conditions in Section 2, its local dimension reflects active state components.

This geometric information differs from spectral peak number: one periodic component can generate many harmonics. Our recurrent experiments show that single-neuron MG separates phase structures with identical peak numbers and similar covariance ranks. Applied experiments test whether the same statistic supports monitoring and decisions from scalar data, alongside strong simple alternatives.

Contributions. We test the information in a scalar record, its practical uses and its computational cost.

1. **Component structure from one scalar.** We define an MG measurement procedure with explicit interpretation conditions. Synthetic and learned recurrent systems test dimension recovery and distinguish independent phases from harmonics.
2. **Model and policy selection from logs.** We test MG against independent measurements in CNNs, VAEs and RL, then use it to select SSL configurations and forecasters. Strong scalar baselines accompany each comparison.
3. **Faster decisions without full-state analysis.** Sampled-query MG is 3.7 times faster than held-out forecast search in the paired benchmark, with no

statistically resolved accuracy difference. It retains all full-MG choices. A separate recurrent benchmark shows 14–19-fold savings over the full Lyapunov spectrum.

The contribution is the measurement procedure and its evaluation; delay embeddings and dimension estimators are established tools.

Related work. Intrinsic-dimension methods analyze point clouds and representations (Facco et al., 2017; Ansuini et al., 2019; Pope et al., 2021); other work studies fractal dimensions of optimizer trajectories and invariant measures (Simsekli et al., 2020; Camuto et al., 2021; Birdal et al., 2021; Tan et al., 2024). Random-walk PCA (Antognini and Sohl-Dickstein, 2018) and Koopman analysis (Redman et al., 2024) help interpret training trajectories. We measure local temporal geometry from one scalar, comparing it with participation ratio, Hessian spectra, and grokking-related changes in function complexity and weight rank (Gao et al., 2017; Cohen et al., 2021; Liu et al., 2023; Yunis et al., 2024; Humayun et al., 2024). Roughness, recurrence, and entropy test whether this geometry adds useful information beyond simpler features.

2 DETECTING SIMPLIFICATION IN NEURAL NETWORK DYNAMICS

2.1 Problem statement

Given a scalar observation $x_t = \phi(s_t)$ of a state s_t , we seek changes in the complexity of its dynamics. During training, s_t contains the P trainable parameters and any optimizer variables. For a trained recurrent network with fixed weights, s_t is its activity. These are distinct trajectories, observed through the same measurement procedure.

In a recurrent regime, let μ describe the long-run fraction of time spent in state-space regions: this is the occupation measure. We assume that this limiting measure exists and is ergodic; finite records do not prove either property. For a Euclidean ball $B(s, \epsilon)$ in a fixed state representation, define

$$d_{\text{act}} = \lim_{\epsilon \rightarrow 0^+} \frac{\log \mu(B(s, \epsilon))}{\log \epsilon}, \quad (1)$$

when the limit exists and is constant μ -almost everywhere. Here ϵ is a positive radius. On a smooth manifold with positive regular density, this exponent equals the manifold dimension; r independent oscillator phases therefore require r local coordinates. Fixed invertible linear rescaling preserves the limit, although finite-scale estimates can change.

During nonstationary training we use a different target: a change in MG that agrees with restricted updates, reduced latent information, or more regular behavior measured independently. Such simplification need not improve performance: losing useful latent information is an undesirable change that a monitor should detect.

2.2 Delay reconstruction and dynamical-systems theory

A single observation can mix several components. Delayed observations can separate them by recording how the signal changes over time. We form the vector

$$y_t = (x_t, x_{t+\tau}, \dots, x_{t+(E-1)\tau}) \quad (2)$$

with E coordinates separated by lag τ . The Takens and Sauer results give conditions under which generic smooth observations distinguish the states of a deterministic system (Takens, 1981; Sauer et al., 1991). Extensions by Stark address forced systems under additional assumptions (Stark, 1999; Stark et al., 2003). These results explain why a scalar record can retain information about the underlying dynamics. They do not prescribe a finite-sample window or guarantee reconstruction for every SGD trajectory.

Our dimension interpretation requires the delay map to be bi-Lipschitz on the sampled support: distances may change only by bounded factors. These factors disappear in the limit in Eq. (1), preserving local dimension. Appendix A gives the statement and proof. We test dimension recovery in recurrent systems with known structure. For applied training, we compare changes in the estimate with independent measures of simplification.

2.3 Pooled local-dimension estimation

Within a window of W scalar samples, we standardize the observations and form $n = W - (E - 1)\tau$ delay vectors. For each vector, we retain k nearest neighbors with time separation greater than the Theiler exclusion T (Theiler, 1986). Let r_{ij} be its Euclidean distance to the j th closest admissible neighbor, ordered increasingly. If every query has enough neighbors, we compute

$$\hat{d}_{\text{MG}} = \frac{n(k-1)-1}{\sum_{i=1}^n \sum_{j=1}^{k-1} \log(r_{ik}/r_{ij})}. \quad (3)$$

The distance ratios measure how quickly neighborhoods fill as their radius grows; faster growth corresponds to higher local dimension. Equation (3) pools this information across query vectors. The subtraction of one is a finite-sample correction under the local likelihood model, which assumes approximately uniform density

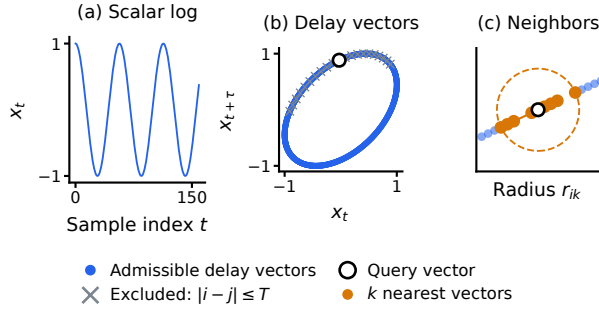


Figure 1: Scalar samples, delay vectors, and neighbor distances. An illustrative periodic signal (a) produces the two-coordinate reconstruction (b). Panel (c) enlarges a neighborhood: blue dots are admissible vectors, gray crosses are temporally excluded, the open circle is the query, and orange dots are its k nearest neighbors. The radius ratios enter Eq. (3).

at the neighborhood scale. Serial dependence, uneven excitation, and short records can still introduce bias. We retain these effects in the reported estimates rather than clipping values to E .

Invariance to affine rescaling. For fixed delay and neighbor settings, the ideal estimator is invariant to $x_t \mapsto ax_t + b$, $a \neq 0$. Standardization removes the offset and amplitude scale, leaving the distance ratios unchanged. Thus MG can distinguish a change in temporal structure from a uniform change in amplitude. Appendix A proves this property and describes the effects of the small numerical noise added in the implementation. The invariance applies to a whole window; a scale change within that window can alter its temporal structure.

Window length and temporal resolution. The delay span $S = (E - 1)\tau$ determines the time interval represented by each vector. The window length W determines how much data enter an estimate and how precisely a transition can be located. Estimating dimension requires repeated visits within one regime. Monitoring requires a window short enough to distinguish the regimes before and after a change. We select settings on development runs and freeze them for evaluation. We also report detection delay. Appendix B lists the configurations.

2.4 Measurement and interpretation pipeline

Applying the estimator requires an observable and a timescale. The following steps connect these choices to the scientific claim being tested.

1. **Choose the scalar and reference outcome.** Specify the event and an independent measurement of it. Select the observable on development runs; record at fixed intervals and measure acquisition cost.
2. **Select window, lag and decision threshold.** Before evaluation, fix the window length W , stride (samples between window endpoints), embedding dimension E , delay τ , number of neighbors k , temporal exclusion T , and decision threshold. Adaptive settings need an explicit rule shared by matched runs.
3. **Compute MG and reject numerical degeneracy.** Use completed windows, dated by their endpoints. Retain raw amplitude and floor flags; check sensitivity to W , τ , and $2E$. Report missing estimates and noise-dominated windows.
4. **Distinguish dimension recovery from change detection.** Dimension estimation requires recurrence, a stable occupation measure, a dimension-preserving observation, and sufficiently many returns above the noise scale. Otherwise, evaluate MG against the reference outcome and scalar baselines, including detection delay and false positives.

Inadequate embedding dimension prevents faithful reconstruction, while finite-sample MG can exceed E . Stable estimates alone do not certify embedding assumptions; changes in SGD noise and temporal correlation can also affect the statistic.

3 RECOVERING COMPONENT STRUCTURE IN SYNTHETIC EXPERIMENTS

We first ask whether MG recovers the component structure predicted by the theory. Analytic signals provide known dimensions; learning problems with known references test the effect of parameter updates; trained generators test whether a single neuron reveals the phase structure of a learned task. Section 4 then evaluates monitoring in applied learning problems.

3.1 Analytic oscillators and learning

Oscillatory signals with known dimension. In the analytic baseline, r diagonal entries of a matrix oscillate at rationally independent frequencies. The trajectory therefore has r independent phases. MG computed from a scalar observation tracks this dimension over $r = 1$ –8, with mean absolute error 0.27. This experiment tests dimension recovery on prescribed oscillations; the following experiments include parameter training.

Classifier training on fixed MLP features. We train a multilayer perceptron (MLP) with tanh activations on handwritten digits (Pedregosa et al., 2011), freeze its features, and train a linear classifier in a fixed parameter subspace. Quasiperiodic changes in the loss weights of data groups provide a known source of recurrent motion. We select estimator settings on separate seeds and ranks, then evaluate ranks 1, 3, 5, 8 on new seeds.

Every usable observable orders the held-out ranks correctly with these fixed settings. Parameter norm and fixed-probe loss stop varying when the learning rate is zero, confirming that their variation requires learning. The mean absolute error against the independently measured covariance reference is 0.90 across rank–seed–observable combinations. This reference checks the excitation of parameter directions; it is not an exact active-dimension label. MG therefore orders the tested ranks without retuning for each rank. Appendix C gives errors for individual observables and distinguishes this task from earlier driven-target tests.

Endogenous oscillations in a regression problem. We next test whether MG detects fewer modes when all parameters remain trainable. We train all 32 weights of a dense linear regressor using full-batch heavy-ball updates. Repeated Hessian eigenvalues form r groups; momentum one produces r independent oscillatory modes with a fixed target. Selective damping reduces four modes to two without changing the data or number of trainable parameters. Across three coordinate/probe seeds, loss-based MG falls from 4.97 to 2.56; these seeds share the same loss dynamics (Appendix C). Multiplying the original loss by 0.01 leaves MG unchanged. The estimate therefore responds to the removal of modes in this experiment, while remaining insensitive to uniform rescaling. Without damping, the optimizer oscillates around the solution rather than converging.

3.2 Learned generators distinguish phases from harmonics

We next test whether a single-neuron recording reveals the phase structure of a trained recurrent network. We train a 1,000-unit rate network by FORCE with recursive least squares (Sussillo and Abbott, 2009) to generate one to four sinusoidal outputs. Feeding the outputs back into the network makes the learned readout affect its dynamics. At evaluation, the weights are fixed and the network runs without the target signal.

We train seven tasks on each of five seeds: one to four independent frequencies (T1–T4), two or four harmonics of one frequency (H2, H4), and two independent frequencies with their harmonics (M4). Their target

Table 1: Learned generator results: medians over five seeds. Phases denotes target dimension; MG uses neuron zero and PR uses the full state. For each seed, MG is the median over four 8,192-sample windows.

Task	Phases	MG	Covariance PR
T1	1	1.15	1.64
T2	2	2.84	3.01
T3	3	5.90	4.32
T4	4	5.93	5.76
H2	1	1.12	2.97
H4	1	1.15	5.27
M4	2	3.58	5.71

phase dimensions are 1, 2, 3, 4, 1, 1, 2, respectively. All 35 networks meet the output-fidelity criterion. A Lyapunov analysis checks the learned dynamics independently. We choose MG settings on the target signals before analyzing the networks and use neuron zero as the primary observable.

The harmonic comparison tests whether MG detects independent phases rather than spectral peaks (Figure 2, Table 1). For every seed, MG is lower for H2 than T2 and increases from H4 to M4 to T4. The latter tasks have identical numbers of spectral peaks and similar covariance PR. MG therefore distinguishes their phase structure from a single-neuron recording. It also separates T1 and T2 from the higher-dimensional tasks, but overestimates dimension for T3 and T4 and does not distinguish them at this record length.

3.3 FORCE training from chaos to periodic output

In a separate FORCE experiment, a 256-unit network learns to produce a figure-eight trajectory from initially chaotic activity. The target has two outputs but one fundamental phase. We select five chaotic initializations using a pretraining Lyapunov criterion, without inspecting MG, and evaluate the network at saved checkpoints.

Single-neuron MG falls from 3.56–5.07 before training to 1.08–1.13 afterward in all five networks. The decline persists across three neurons and the tested window and lag settings. Independently, the full Lyapunov spectrum loses its clearly positive exponents. Four networks meet the stricter stable-cycle criterion; the fifth has weak contraction transverse to the cycle.

Both analyses identify simplification of the learned activity. MG does so from one neuron, while the spectrum uses the full state and tangent dynamics. Section 5 compares their computational costs. Here the measured trajectory is network activity at fixed weights, not the sequence of weight updates.

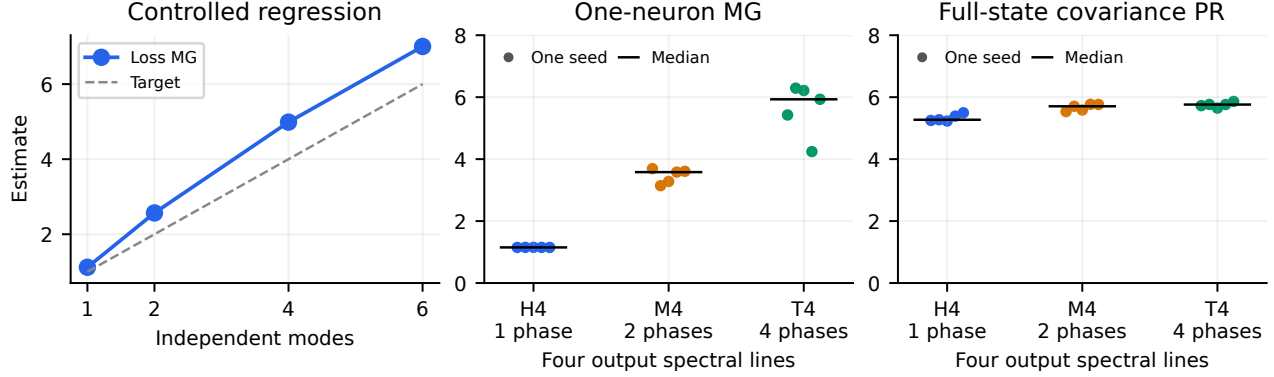


Figure 2: Phase structure recovered from scalar records. Left: regression with known modes; the dashed line is the target dimension. Center and right: generators with four output spectral lines and one (H4, blue), two (M4, orange), or four (T4, green) target phases. Dots represent seeds and black bars indicate medians. Single-neuron MG separates these phase structures despite similar full-state covariance participation ratios (PR).

4 MONITORING AND SELECTION IN LEARNED SYSTEMS

The applications test two uses of the same scalar statistic: monitoring independently measured changes and making decisions without access to full trajectories. CNN, VAE and grokking experiments address monitoring; policy, SSL and forecast selection address decisions. These tests establish empirical utility separately from absolute dimension recovery.

4.1 Detecting restrictions on CNN parameter updates

We train a 14,666-parameter convolutional neural network (CNN) on 10,000 CIFAR-10 images (Krizhevsky, 2009) for 10,000 SGD updates. At update 4,000, we reduce the learning rate, freeze layers, or prune weights. Reference runs continue unchanged or use a larger batch; separate transformed logs rescale or smooth the observation without altering training. MG receives only the norm of all parameters, which requires no extra network forward pass. Settings are $W = 1,000$, stride 500, $E = 20$, $\tau = 1$, and $k = 20$.

We choose detector settings on seeds 0–3 and test interventions of different strengths on new seeds 10–13. Independent measurements track update covariance participation ratio, the fraction of moving parameters and update magnitude. Freezing and pruning impose known restrictions on trainable or nonzero parameters. Learning-rate reduction changes the dynamics, but need not reduce invariant dimension; we assess its effect through the measured updates.

MG falls after strong restrictions on training. Freezing all but the classification head reduces it by a median 18.5%, while update participation ratio falls by 43.5%.

Strong freezing and pruning produce median MG declines of approximately 12–19%; reference-run medians range from near zero to increases of 13%. Across intervention strengths, MG ratios and update participation-ratio changes have Spearman correlation 0.59. The two statistics need not agree: after 50% pruning, MG falls while update participation ratio rises.

MG separates interventions from unchanged-training and increased-batch reference runs with AUC 0.93, versus 0.48–0.57 for trend crossings, lag-one correlation, and detrended standard deviation. IAAFT spectral surrogates yield 0.68. Appendix D.2 specifies the descriptive surrogate check, all scalar competitors, and alternative comparison sets.

Causal detection of training interventions. For window estimates m_j , we compare the median over the latest M windows with the median over the preceding B windows:

$$D_j = \frac{\text{median}(m_{j-M+1}, \dots, m_j)}{\text{median}(m_{j-M-B+1}, \dots, m_{j-M})} - 1. \quad (4)$$

We choose $M = 2$, $B = 3$, and a detection threshold of $D_j < -0.0883$, a drop of 8.83%. Only completed windows enter the rule. The first threshold crossing defines a detected change. It is correct if it occurs within 5,000 updates after the intervention; an earlier crossing is a false positive. Competing statistics use the same selection procedure.

MG detects 27/28 strong interventions with 0/16 reference false positives, while normalized increments give 28/28 and 0/16 (Table 2). MG detects 8/28, 16/28, and 22/28 within 500, 1,000, and 2,000 updates. On ten additional seeds with shifted intervention times, it gives 19/30 detections and false positives in five of ten reference runs, versus 30/30 and zero for absolute

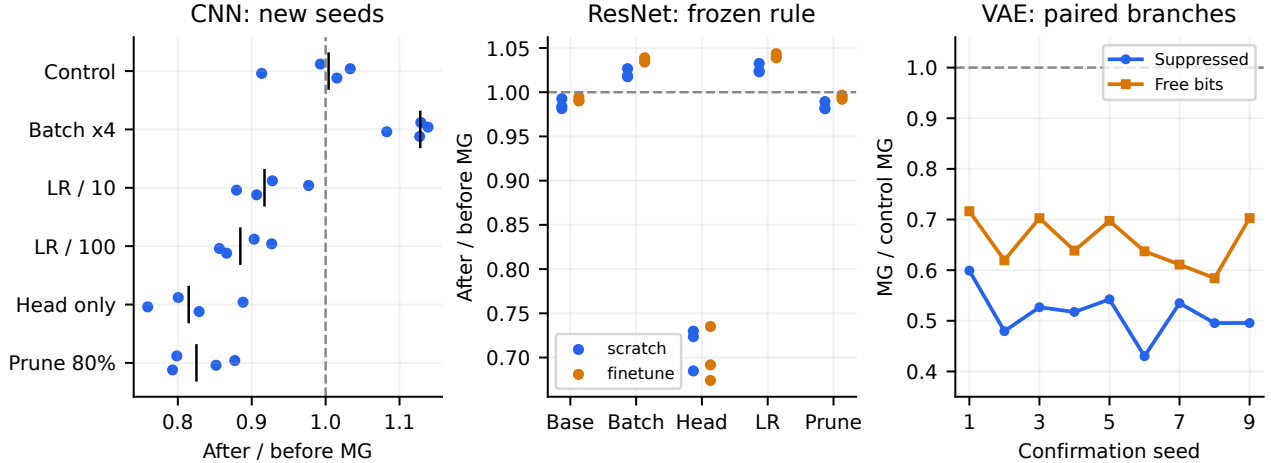


Figure 3: MG under training interventions and latent-information suppression. Left: post/pre-intervention ratios on four CNN seeds; vertical marks indicate medians. Center: corresponding ResNet-18 ratios for scratch and pretrained initialization. Right: late VAE estimates relative to the shared low-regularization reference run, for suppression and free-bits branches on nine seeds. Points represent seeds; dashed lines indicate a ratio of one. Independent update and information measurements provide the references.

Table 2: All scalar features with the same fixed block detector. Hits use a 5,000-update horizon. False positives are reference records with any threshold crossing; delay is the median over detected events. Four held-out seeds are shared across interventions and reference runs.

Feature	Detected	FP	Delay
MG	27/28	0/16	1,000
Abs. increments	27/28	0/16	500
Norm. increments	28/28	0/16	1,500
Perm. entropy	28/28	7/16	1,500
Sample entropy	28/28	4/16	1,500
Spectral entropy	4/28	2/16	2,750
Trend crossings	15/28	5/16	1,000
Lag-one corr.	0/28	0/16	—
Detrended std.	28/28	10/16	500
Level	4/28	0/16	1,000

increments. The weaker result on shifted intervention times limits transfer of the original threshold. Runs sharing a seed are paired observations; Appendix D.2 gives the comparison details.

4.2 Transferring the detector to ResNet-18

We transfer the parameter-norm observable, MG settings, and detection threshold to ResNet-18 (He et al., 2016), without retuning. Training uses the full CIFAR-10 training set, either from scratch or from ImageNet-pretrained weights, with three new seeds in each setting. Freezing the backbone while continuing to train the head reduces MG by a median 27.6% and 30.8%, respectively. The detector identifies all six freezing events

with no false positives in 24 reference records.

The freezing result transfers from about 15 thousand to 11 million parameters. The same rule misses learning-rate reductions and pruning; layer norms and random projections also introduce false positives. Transfer therefore depends on the event and observable.

4.3 Tracking latent information in a text VAE

A text variational autoencoder (VAE) learns to reconstruct Penn Treebank sentences (Marcus et al., 1993) through a 16-dimensional Gaussian latent code. Its encoder and decoder use gated recurrent units (Bowman et al., 2016); the model has 276,016 parameters and trains on 6,000 sentences. We test whether MG detects reduced dependence of decoder outputs on the latent code.

After 1,024 shared updates with Kullback–Leibler (KL) regularization weight $\beta = 0.01$, we split training into three branches: keep that weight, raise it to one, or use $\beta = 1$ with a free-bits objective. The free-bits objective excludes low-KL latent coordinates from further KL penalization. All branches train until update 3,072.

MG uses the reconstruction negative log-likelihood (NLL) per token on 16 fixed evaluation sentences, with fixed latent sampling noise. It has no access to KL or the independent references. On a larger set of 256 fixed evaluation sentences, the references estimate latent mutual information and the change in decoder outputs after shuffling codes between sentences. A joint decline means that less information is retained and used

through the latent code.

The information-loss criterion is met on all nine test seeds. MG declines more strongly in the high-regularization branch than in the matched reference on every seed: their median late MG ratio is 0.517, with a 95% seed-bootstrap interval [0.480, 0.542]. Free bits partially preserves both information and decoder dependence. MG is then higher than in the suppressed branch on all nine seeds, with median ratio 1.286 [1.179, 1.418]. The two branches use the same KL weight but retain different amounts of information. MG distinguishes them from reconstruction logs alone.

Standard deviation and spectral entropy also detect information loss on all nine seeds. Using MG to schedule reference checks is less successful: a separate cyclic-VAE test finds 28/41 transitions, versus 38/41 for periodic checks. Tracking information loss and scheduling measurements are distinct tasks.

4.4 Detecting changes during grokking

Parameter-norm MG falls near generalization in four modular-arithmetic runs when its windows match the resolution of stored trajectory sketches. Those sketches independently show a reversible fall in detrended covariance participation ratio. Across analysis settings, 24/26 usable comparisons separate these runs from two references, but reuse the same trajectories. Roughness changes are stronger. A memorizing run also simplifies, while the full-batch setting does not reproduce the effect. MG detects a transition here; its decrease is not specific to generalization.

4.5 Selecting Walker2d policies under actuator shifts

PPO policies in Walker2d (Schulman et al., 2017; Todorov et al., 2012) show lower action-log MG under moderate action smoothing in two five-seed series. This does not imply simpler whole-body motion. We additionally select among 16 saved policies per seed using three nominal rollouts and a shared reward/survival gate. On four held-out fine-tuning seeds, minimum action-norm MG gives deployment return 4.313 versus 4.005 for nominal-reward selection under hidden actuator lag/noise; completion rises from 80.0% to 90.8%. Spectral entropy performs better (4.480, 97.5%), and a pilot-selected fixed configuration nearly matches MG (4.310). This is exploratory policy selection, not demonstrated curriculum or training acceleration; Appendix E.1 reports all comparisons and shared-initial-policy limitations.

4.6 Selecting SSL configurations from gradient logs

Joint-embedding self-supervised learning (SSL) trains representations to agree across augmented views. Agreement alone can admit collapsed solutions that map different images to the same representation (Chen and He, 2021); training loss is therefore an unreliable guide to representation quality (Garrido et al., 2023). We test whether gradient-norm MG can guide configuration choice without extracting embeddings from each evaluated model. The MNIST experiment compares 29 configurations across four objectives using a 784–256–256 MLP encoder. We choose the log and score direction from labeled development runs, then freeze the rule for five test seeds. Applying it requires no labels.

MG selects configurations with low average regret and ranks them more accurately than encoder RankMe, the tested α -ReQ slope and LiDAR implementations (Garrido et al., 2023; Agrawal et al., 2022; Thilak et al., 2024). Table 22 reports correlations and selection regret. Embedding spread has the highest mean correlation; several simple scalar features remain competitive. The benefit is useful selection from a saved gradient log without an embedding pass. Appendix I details the single-dataset scope, proxy implementations and uncertainty.

4.7 Warning of loss spikes in transformer training

In a two-layer transformer trained on modular addition, MG of log-loss warns of most test spikes within the specified horizon. It has higher AUROC than the tested attention-logit and attention-entropy monitors. Roughness detects more events at the same false-positive rate for less computation (Table 25). This result supports detecting the slingshot regime from scalar logs. Preventing spikes and transferring the rule to language models remain untested (Appendix I).

4.8 Forecast model selection from single-neuron recordings

Can a scalar record identify a suitable forecasting model without fitting every candidate to held-out data? We train a selector that uses MG from 2,048 observations of one neuron to choose among eight forecasters. The selected model predicts the next 32 observations. Candidates include persistence, mean and cycle forecasts, linear regression, nearest-neighbor prediction and a nonlinear feature model. We fit the selector on 18 initialization seeds and freeze it for evaluation. Networks that miss their training targets remain included; the

Table 3: Forecast selection on 20 new seeds, eight tasks per seed. NMSE uses prefix variance. Time includes selection and the chosen fit on 12 matched inputs; a dash means not benchmarked. Acquisition and offline selector training are excluded.

Selector	NMSE	Time (ms)
MG	0.0364	199.1
MG, $q = 128$	0.0364	12.2
Held-out model search	0.0407	45.3
Cheap + held-out search	0.0338	—
Held-out errors + MG	0.0317	—

Table 4: Analysis time, excluding scalar acquisition. References: full Lyapunov spectrum (FORCE; pre/post-training medians) and information/code-shuffling measurements (VAE). Comparisons are within rows; Appendix F documents hardware and timing procedures.

Setting	MG	Reference
CNN, 1,000-point window	9.5 ms	—
FORCE, 256 units	0.60–0.84 s	11.55–12.01 s
VAE, 512-point window	12.89 ms	209.32 ms

task is to predict the activity they actually produce (Appendix G).

MG has lower mean NMSE than TwoNN and Levina–Bickel selection; the difference from held-out search is unresolved (Table 19). A query subsample preserves every full-MG choice on further runs at lower cost (Table 3). Selectors using held-out forecast errors can be more accurate. The saving concerns selection from an available record.

5 COMPUTATIONAL COST AND MEASUREMENT REQUIREMENTS

MG analyzes one scalar per step, with storage and neighbor-search cost set by W and E . It needs no model access once the record is available. Acquisition is separate: a parameter or gradient norm requires an $O(P)$ reduction; probe loss requires a forward pass.

The CNN scalar record occupies about 39 KiB, compared with 559 MiB for the stored float32 parameter trajectory (Table 4).

Comparison with full-state diagnostics. In the FORCE benchmark, MG identifies the transition toward a cycle from one neuron; the full Lyapunov spectrum independently confirms the change using state and tangent dynamics. Table 4 measures the cost of these two analyses, not two equivalent outputs: MG does

not recover individual exponents. A largest-exponent calculation and simple scalar features can be cheaper. This saving matters when a scalar summary is sufficient (Appendix F).

Cost of obtaining the scalar. The forecast experiment uses an available record, so fewer MG queries directly reduce selection cost (Section 4.8). In the cyclic VAE experiment, probe evaluations dominate. Low analysis cost is most useful when the scalar is already recorded.

Observable sensitivity and detector transfer. The CNN detector transfers to backbone freezing, but performs worse than trend crossings in the ReLU-death test. MG also fails to track near-threshold Hessian modes in the tested edge-of-stability setting. Each new observable and event therefore needs a separate check (Section 2.4). Noise, drift and numerical effects in nearly constant records can alter the estimate.

6 CONCLUSION

We developed a method for extracting geometric information from one scalar log. In recurrent experiments, single-neuron MG distinguishes phase structures with identical spectral peak numbers and similar covariance ranks. In applied learning, it tracks independently measured changes and guides model selection without state trajectories or embedding extraction on evaluated runs. Sampled queries make forecast selection faster than held-out model search in our benchmark. These results connect scalar observations to useful information about dynamical complexity. Exact dimension recovery requires the stated assumptions; empirical utility remains task-dependent.

AI USE STATEMENT

The authors formulated the research questions, hypotheses, and scientific claims. Generative AI tools assisted with conducting experiments, including implementation, debugging, analysis, and preparation of figures and reports. They also assisted with drafting, translation, and editing. The authors checked all reported results, read the complete manuscript, and further revised the text. The authors take responsibility for the final text, claims, code, and experimental results.

REFERENCES

- Kumar K. Agrawal, Arnab Kumar Mondal, Arna Ghosh, and Blake Richards. α -ReQ: Assessing representation quality in self-supervised learning by measuring eigenspectrum decay. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Alessio Ansuini, Alessandro Laio, Jakob H. Macke, and Davide Zoccolan. Intrinsic dimension of data representations in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Joseph M. Antognini and Jascha Sohl-Dickstein. PCA of high dimensional random walks with comparison to neural network training. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.
- Christoph Bandt and Bernd Pompe. Permutation entropy: a natural complexity measure for time series. *Physical Review Letters*, 88(17):174102, 2002. doi: 10.1103/PhysRevLett.88.174102.
- Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=xm6YD62D1Ub>.
- Tolga Birdal, Aaron Lou, Leonidas Guibas, and Umut Simsekli. Intrinsic dimension, persistent homology and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.
- Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew M. Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 10–21, 2016. doi: 10.18653/v1/K16-1002.
- Alexander Camuto, George Deligiannidis, Murat A. Erdogdu, Mert Gurbuzbalaban, Umut Simsekli, and Lingjiong Zhu. Fractal structure and generalization properties of stochastic optimization algorithms. *arXiv preprint arXiv:2106.04881*, 2021.
- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pages 1597–1607. PMLR, 2020.
- Xinlei Chen and Kaiming He. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15750–15758, 2021.
- Jeremy M Cohen, Simran Kaur, Yuanzhi Li, J Zico Kolter, and Ameet Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=jh-rTtvkGeM>.
- Elena Facco, Maria d’Errico, Alex Rodriguez, and Alessandro Laio. Estimating the intrinsic dimension of datasets by a minimal neighborhood information. *Scientific Reports*, 7(1):12140, 2017.
- Peiran Gao, Eric Trautmann, Byron M. Yu, Gopal Santhanam, Stephen Ryu, Krishna Shenoy, and Surya Ganguli. A theory of multineuronal dimensionality, dynamics and measurement. *bioRxiv* 214262, 2017. doi: 10.1101/214262.
- Quentin Garrido, Randall Balestrieri, Laurent Najman, and Yann LeCun. RankMe: Assessing the downstream performance of pretrained self-supervised representations by their rank. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 10929–10974. PMLR, 2023. URL <https://proceedings.mlr.press/v202/garrido23a.html>.
- Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 21271–21284, 2020.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- Ahmed Imtiaz Humayun, Randall Balestrieri, and Richard Baraniuk. Deep networks always grok and here is why. *arXiv preprint arXiv:2402.15555*, 2024.

- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Elizaveta Levina and Peter J. Bickel. Maximum likelihood estimation of intrinsic dimension. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 17, 2004. URL <https://proceedings.neurips.cc/paper/2004/file/74934548253bcab8490ebd74afecebb5-Paper.pdf>.
- Ziming Liu, Eric J. Michaud, and Max Tegmark. Omnigrok: Grokking beyond algorithmic data. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=zDiHoIWa0q1>.
- David J. C. MacKay and Zoubin Ghahramani. Comments on ‘maximum likelihood estimation of intrinsic dimension’ by E. Levina and P. Bickel, 2005. URL <https://www.inference.org.uk/mackay/dimension/>. Unpublished note.
- Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2):313–330, 1993.
- E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1–2):100–115, 1954. doi: 10.1093/biomet/41.1-2.100.
- Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Phil Pope, Chen Zhu, Ahmed Abdelkader, Micah Goldblum, and Tom Goldstein. The intrinsic dimension of images and its impact on learning. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=XJk19XzGq2J>.
- William T. Redman, Juan Bello-Rivas, Maria Fonoberova, Ryan Mohr, Yannis G. Kevrekidis, and Igor Mezic. Identifying equivalent training dynamics. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. doi: 10.52202/079017-0744.
- Joshua S. Richman and J. R. Moorman. Physiological time-series analysis using approximate entropy and sample entropy. *American Journal of Physiology–Heart and Circulatory Physiology*, 278(6):H2039–H2049, 2000. doi: 10.1152/ajpheart.2000.278.6.H2039.
- Tim Sauer, James A. Yorke, and Martin Casdagli. Embedology. *Journal of Statistical Physics*, 65(3–4):579–616, 1991.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.
- Umut Simsekli, Ozan Sener, George Deligiannidis, and Murat A. Erdogdu. Hausdorff dimension, heavy tails, and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 5138–5151, 2020.
- Jaroslav Stark. Delay embeddings for forced systems. I. Deterministic forcing. *Journal of Nonlinear Science*, 9(3):255–332, 1999. doi: 10.1007/s003329900072.
- Jaroslav Stark, David S Broomhead, ME Davies, and J Huke. Delay embeddings for forced systems. II. Stochastic forcing. *Journal of Nonlinear Science*, 13(6):519–577, 2003. doi: 10.1007/s00332-003-0534-4.
- David Sussillo and L. F. Abbott. Generating coherent patterns of activity from chaotic neural networks. *Neuron*, 63(4):544–557, 2009. doi: 10.1016/j.neuron.2009.07.018.
- Floris Takens. Detecting strange attractors in turbulence. In *Dynamical systems and turbulence, Warwick 1980*, pages 366–381. Springer, 1981. doi: 10.1007/BFb0091924.
- Charlie B. Tan, Ines Garcia-Redondo, Qiquan Wang, Michael M. Bronstein, and Anthea Monod. On the limitations of fractal dimension as a measure of generalization. *arXiv preprint arXiv:2406.02234*, 2024.
- James Theiler. Spurious dimension from correlation algorithms applied to limited time-series data. *Physical Review A*, 34(3):2427–2432, 1986.
- Vimal Thilak, Etai Littwin, Shuangfei Zhai, Omid Saremi, Roni Paiss, and Joshua M Susskind. The slingshot mechanism: An empirical study of adaptive optimizers and the loss landscape. *arXiv preprint arXiv:2206.04817*, 2022. URL <https://arxiv.org/abs/2206.04817>.
- Vimal Thilak, Chen Huang, Omid Saremi, Laurent Dinh, Hanlin Goh, Preetum Nakkiran, Joshua M. Susskind, and Etai Littwin. LiDAR: Sensing linear probing performance in joint embedding SSL architectures. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=f3g5XpL9Kb>.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012.
- David Yunis, Kumar Kshitij Patel, Samuel Wheeler, Pedro Henrique Pamplona Savarese, Gal Vardi, Karen Livescu, Michael Maire, and Matthew Walter. Grokking, rank minimization and generalization in deep learning. In *ICML 2024 Workshops: MI*, 2024.

CHECKLIST

1. For all models and algorithms presented, check if you include:
 - (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [Yes] Sections 2–4 and the appendices.
 - (b) An analysis of the properties and complexity (time, space, sample size) of any algorithm. [Yes] Scale invariance, window requirements, storage and empirical cost are discussed; no universal sample-complexity bound is claimed.
 - (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [No] The bundle includes analysis scripts and numerical evidence, but not the complete training software with all dependencies.
2. For any theoretical claim, check if you include:
 - (a) Statements of the full set of assumptions of all theoretical results. [Yes] The propositions state their assumptions. Embedding results are cited as motivation and are not claimed to hold for every training record.
 - (b) Complete proofs of all theoretical results. [Yes] Appendix A proves the two elementary propositions stated here; established embedding theorems are cited.
 - (c) Clear explanations of any assumptions. [Yes] Section 2 and the supplementary material distinguish geometric interpretation from empirical monitoring.
3. For all figures and tables that present empirical results, check if you include:
 - (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [No] Saved outputs and scripts reproduce tables and figures; the materials required to repeat every training run are not fully bundled.
 - (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [Yes] Experiment-specific descriptions and development splits are in the appendices and accompanying procedures.
 - (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [Yes] Captions and Appendix B specify statistics, replication units, and the VAE bootstrap.
 - (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [No] Appendix F.1 documents recovered CPU, software, and thread settings; hardware metadata are incomplete for some historical runs.
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
 - (a) Citations of the creator If your work uses existing assets. [Yes] Dataset/model and methodological sources are cited in the text and asset notes.
 - (b) The license information of the assets, if applicable. [No] A consolidated license inventory is not included in this version.
 - (c) New assets either in the supplemental material or as a URL, if applicable. [Yes] Numerical evidence and manuscript-reconstruction scripts are provided with the source bundle.
 - (d) Information about consent from data providers/curators. [Not Applicable] No newly collected participant data are used.
 - (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [Not Applicable] No new textual or image dataset is released; the bundled evidence is numerical.
5. If you used crowdsourcing or conducted research with human subjects, check if you include:
 - (a) The full text of instructions given to participants and screenshots. [Not Applicable]
 - (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Not Applicable]
 - (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Not Applicable]

A THEORY, IMPLEMENTATION, AND EXPERIMENTAL DETAILS

A.1 Dimension preservation under delay reconstruction

The dimension interpretation has two steps. First, delay-embedding theorems give conditions under which a generic scalar observable can reconstruct the state of a deterministic system. Second, a local dimension estimator approximates the geometry of that reconstruction from finite data. The reconstruction theorem alone does not guarantee accurate estimation; the estimator alone does not establish an embedding.

For a stationary ergodic trajectory, the occupation measure can be written as $\mu(A) = \lim_{N \rightarrow \infty} N^{-1} \sum_{t=1}^N \mathbf{1}\{s_t \in A\}$ on sets whose boundary has zero measure. It records long-run occupancy, rather than the dimension of a finite list of observations. In Eq. (1), $\epsilon \rightarrow 0^+$ and $\epsilon \downarrow 0$ denote the same right-hand limit. We use the former to emphasize that ϵ is a positive distance scale, distinct from embedding dimension E . Nonstationary training need not have such a limiting measure; the applied experiments therefore use finite-window scores and external reference outcomes.

Proposition 1 (Bi-Lipschitz invariance). *Let μ have local dimension d at s , and let F be bi-Lipschitz on its support, with constants $0 < c \leq C < \infty$. Then the pushforward $F_{\#}\mu$ has local dimension d at $F(s)$.*

Proof. For sufficiently small ϵ , the inverse image of $B(F(s), \epsilon)$ is bounded on the support between $B(s, \epsilon/C)$ and $B(s, \epsilon/c)$. Their probabilities bound the pushforward probability. The logarithms of c and C contribute vanishing constants after division by $\log \epsilon$, so both bounds have the same limiting exponent d . $\square$

This property explains why bounded distance distortion preserves local dimension. A smooth embedding of a compact smooth manifold provides the needed local regularity. For a fractal support, an injective map alone need not provide the required bounds on distance distortion. The forced-system results of Stark (1999); Stark et al. (2003) impose additional conditions and include fibre-wise formulations. Their application to a stochastic optimizer would require checking those conditions; it does not follow from observing a scalar training log.

A.2 Recurrent, transient, and stochastic regimes

Figure 4 makes the assumptions operational. A recurrent trajectory returns to nearby states, a transient moves through a region without repeated visits, and stochastic forcing fills the observed space according to the noise and the observation channel. The same numerical estimator returns a value in all three cases; the regime determines whether that value can be called a dimension.

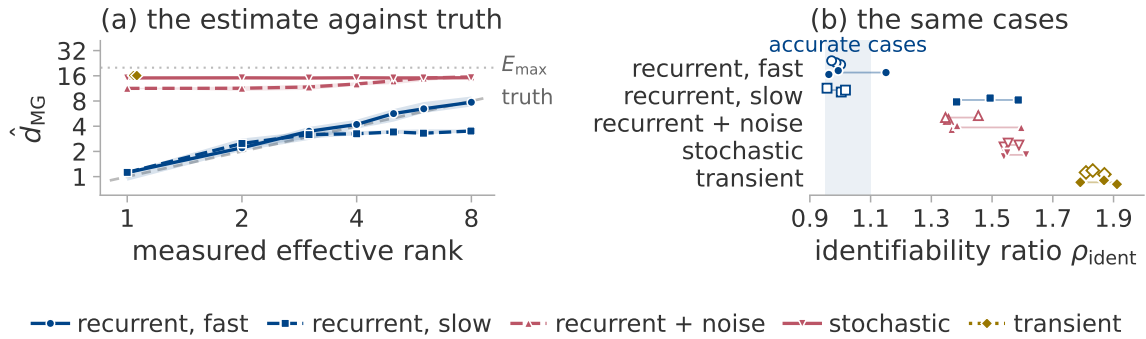


Figure 4: Regime diagnostics from the synthetic study. The left panel compares MG with an independently measured effective rank; the right panel separates recurrent and non-recurrent records using identifiability. The shaded region marks the tested regime in which the component interpretation is supported.

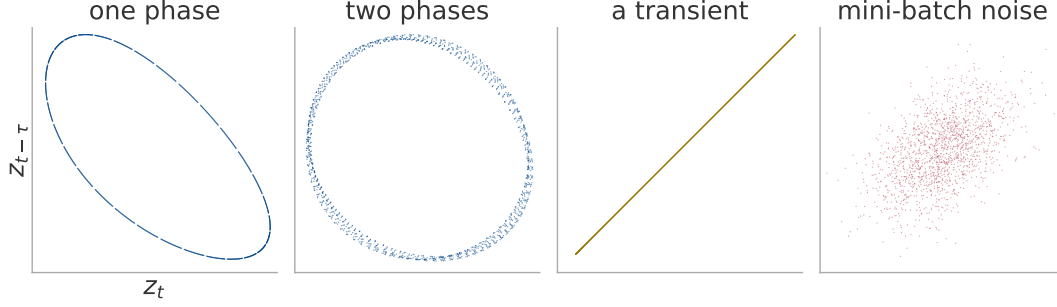


Figure 5: Delay reconstructions of four cases. A recurrent one-phase signal closes a loop; two phases fill a higher-dimensional set; a transient follows a path without returning; stochastic forcing fills the available region. These pictures explain why recurrence and identifiability are required before reading MG as active dimension.

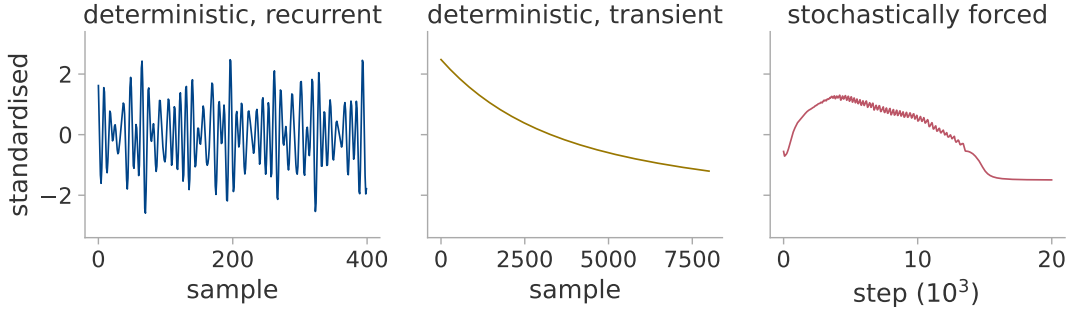


Figure 6: The same scalar-log viewpoint across regimes: recurrent forcing, a transient optimization path, and a stochastically forced training record. The estimator is applied to all three, while the interpretation changes with the regime.

A.3 Computing the pooled estimator

For reproducibility, the implementation can be summarized as follows. The dither and numerical floors only handle ties and degeneracy; they do not provide a statistical correction. The implementation standardizes first, adds Gaussian dither with scale 10^{-9} , floors distances at 10^{-8} and local sums at 10^{-5} , and marks a window when more than 1% of either quantity reaches a floor.

Input: scalar record $x_{1:N}$, lag τ , embedding dimension E , number of neighbors k , Theiler exclusion T .

1. Reject nonfinite, constant, or too-short records. Standardize x_t and add fixed-scale dither, giving z_t .
2. Form $y_t = (z_t, z_{t+\tau}, \dots, z_{t+(E-1)\tau})$.
3. For each y_i , find the k closest y_j with $|i - j| > T$. Return an unavailable estimate if neighbors are insufficient.
4. Accumulate $S_i = \sum_{j=1}^{k-1} \log(r_{ik}/r_{ij})$, recording distance and sum floors.
5. Set $n = N - (E - 1)\tau$, the number of valid delay vectors. Return $\hat{d}_{MG} = (n(k - 1) - 1) / \sum_{i=1}^n S_i$ and the degeneracy flags.

The change detector is separate from the estimator: it compares recent and earlier blocks of window estimates. This separation lets us compare MG with other scalar features under the same causal decision rule.

A.4 Affine invariance and its implementation boundary

Proposition 2 (Affine invariance of the ideal scalar estimate). *For a finite nonconstant record, fixed delay and neighbor settings, and no numerical dither, the MG formula is unchanged by replacing every observation with $ax_t + b$, for any $a \neq 0$, provided the estimate is defined.*

Table 5: Primary configurations. An exclusion of “span” is $(E - 1)\tau$. The adaptive generator lag is one quarter of the autocorrelation time, capped so the delay span occupies at most one eighth of the window; its exclusion is capped at 320. Alternative settings are reported as sensitivity checks, not substituted for the primary results.

Experiment	Observable	W	E	τ	k	Theiler exclusion	Stride
Digits head	state-based	8,000	20	4	20	span; cap 150	3,000
Regression	loss / probe	2,048; 8,192	16	1	10	100	endpoints
Generator	neuron zero	8,192	20	adaptive	20	ACF; cap 320	8,192
CNN / ResNet-18	parameter norm	1,000	20	1	20	span	500
VAE	probe NLL	512	20	1	20	39	128
Walker2d	joint / action scalar	2,048	20	8	20	312	512 / 1,024

Proof. After centering and dividing by standard deviation, the transformed series is the original standardized series multiplied by $\text{sign}(a)$. All pairwise Euclidean distances of its delay vectors are identical. Temporal exclusions and admissible neighbors therefore remain unchanged, as do all distance ratios and their pooled estimate. $\square$

The implemented code adds Gaussian dither of scale 10^{-9} after standardization, floors distances at 10^{-8} and local log-ratio sums at 10^{-5} , and records the fractions affected by these floors. Positive affine changes preserve the standardized input in exact arithmetic and preserve a run with the same dither. A negative rescaling preserves the distribution under symmetric dither, but need not give exactly equal values with the same realized dither. Finite precision and almost-constant records require separate checks. The proposition concerns a whole-window affine transformation: a scale change within a window is a different time series.

A.5 Pooling and dependence

The numerator $n(k - 1) - 1$ is the bias correction used by the existing implementation. Its Gamma-model motivation assumes idealized independent local statistics. Neighbor reuse and serial dependence mean that this correction is not a general finite-sample unbiasedness guarantee for the records in this paper. We use the same implementation throughout, preserve degeneracy flags, and do not clip large estimates to E . The Theiler exclusion removes temporally adjacent candidates but does not make the remaining observations independent.

B CONFIGURATIONS AND REPRODUCIBILITY

Tables and figures are generated from saved CSV/JSON experimental outputs. The accompanying source bundle records file hashes and maps numerical claims to their sources. We specify the observable, estimator settings, and development/test split for each task. All tasks use the same estimator; window length and lag are chosen for the relevant timescale.

Exact adaptive rules. For centered samples z_t , define $a(\ell) = \sum_{t=1}^{W-\ell} z_t z_{t+\ell} / \sum_{t=1}^W z_t^2$. The autocorrelation time t_a is the first lag with $a(\ell) < c^{-1}$, or W if none exists. The adaptive generator uses

$$\begin{aligned}\tau_0 &= \max(1, \text{round}(t_a/4)), \\ \tau &= \min\{\tau_0, \max(1, \lfloor W/[8(E - 1)] \rfloor)\}.\end{aligned}$$

The implementation uses nearest-even rounding. Theiler settings resolve to $T = \min(T_{\max}, T_0)$, where $T_0 = (E - 1)\tau$ for the embedding rule and $T_0 = \max((E - 1)\tau, t_a)$ for the autocorrelation rule. An explicit integer is also capped. Thus a cap can shorten the requested exclusion below the embedding span; report both the rule and its resolved value. The primary digits configuration uses the embedding rule (76 samples), not an uncapped autocorrelation rule.

The historical digits sweep uses stride 3,000 on the 26,000 post-burn samples ($\max(500, \lfloor (N - W)/6 \rfloor)$); development used stride 2,000. The known-mode table uses endpoint windows; its additional switch trace uses $W = 4,096$ and stride 2,048. Walker uses stride 512 in the early repair series and 1,024 in later three-window summaries. Generator windows are disjoint. The cyclic VAE uses stride 64. These variations are fixed for each procedure, not selected after comparing outcomes.

Window length and recurrence requirements. A window must leave enough admissible neighbors after delay reconstruction and temporal exclusion. Meeting the software minimum is necessary but does not guarantee an accurate estimate. For dimension estimation, the window should contain repeated visits within one regime. The lag must also expose the phase structure: increasing record length cannot compensate for delay vectors that cover only a nearly linear segment. For detection, longer windows stabilize estimates but blur transitions. We report detection delay to make this tradeoff explicit.

Units of replication. The CNN interventions share seeds and pre-intervention histories; rescaling and smoothing are transformations of unchanged logs. Generator tasks reuse initialization seeds. The VAE branches share initial trajectories, data, and probes. The Walker continuations share a learned starting policy. We report detection totals and paired contrasts at these levels; we do not treat overlapping windows or transformed logs as independent draws from a population. The VAE intervals are percentile bootstrap intervals over nine paired confirmation seeds, using 20,000 resamples and bootstrap seed 20260929. They measure initialization/batch variation conditional on the selected data and observable.

C DEVELOPMENT SYSTEMS AND MLP-BASED RECOVERY

C.1 Analytic and driven-learning evaluation

The synthetic suite includes an analytic matrix with independent oscillatory entries and learning systems with time-dependent targets or forcing: online regression, logistic regression, a frozen nonlinear decoder, and an MLP in a parameter subspace. To distinguish input variation from learned motion, we set the learning rate to zero and check whether each observable still changes. Several early loss and gradient observables continue varying. Their dimension estimates can recover the forcing structure, but cannot be attributed solely to changing parameters. The digits-head experiment below addresses this ambiguity with state-based observables that become constant when learning stops.

Table 6: Synthetic results against the specified references. Mean absolute error (MAE) averages rank–seed–observable combinations. The driven-target MLP signal still varies with frozen parameters; its error describes the driven signal, not parameter motion alone.

Construction	Range	MAE
Analytic oscillating matrix	1–8	0.27
Driven-target MLP subspace	1–20	0.92
Digits head, covariance reference	1–8	0.90

In the digits experiment, we train and freeze an MLP feature extractor, then train a linear head in a fixed affine parameter subspace. The cross-entropy loss assigns weights to twelve fixed data groups. For the historical digits experiment, weights are $w_i(t) = \max\{0.05, 1 + 0.8 \sum_{j=1}^r G_{ji} \sum_{\ell=1}^r A_{j\ell} \sin(2\pi f_\ell t + \phi_\ell)\}$. Here G identifies the twelve fixed data groups and A equalizes their linearized forcing directions and gains. For $r > 1$, $f_j = 2^{a_j}/16$, with $a_j = 0.03 + 0.94(b_j - \min b)/(\max b - \min b)$ and $b_j = \text{frac}(\sqrt{p_j})$ for the first r primes; for $r = 1$, $f_1 = \sqrt{2}/16$. Phases are uniform on $[0, 2\pi)$ from NumPy’s generator seeded by $31 + \text{seed}$; group assignment uses $7717 + \text{seed}$. The drive amplitude is 0.8 and the preconditioned head step is 0.15. These describe the archived implementation behind the reported scores; the newer code organizes random streams differently. Development uses seeds 90–92 and ranks 2,4,6; evaluation uses seeds 0–3 and held-out ranks 1,3,5,8. The selected settings are $E = 20$, $\tau = 4$, $k = 20$, and an 8,000-point window. This configuration lies on the boundary of the search grid and is held fixed for evaluation; the search does not establish a global optimum.

All usable state-based observables order the held-out ranks correctly. The mean error against measured covariance effective rank is 0.90 across individual records; aggregating by rank before computing the error gives 0.382. We report the former because these aggregations answer different questions. Errors by observable are 0.60 for fixed-probe loss, 0.41 for parameter norm, 0.34 for a fixed parameter projection, and 1.81 for gradient norm. Instantaneous mini-batch loss continues varying with frozen parameters, while quantized probe accuracy is degenerate; neither supports the parameter-dimension interpretation. The specified phases define the intended structure, and the covariance reference checks which directions are measurably excited.

C.2 Dense regression with endogenous modes

The data matrix has 256 rows and 32 columns, full column rank, and Hessian $H = X^\top X/256$. Orthogonal rotations make the data and learned weights dense. Eigenvalues are grouped into r repeated values, with a single excited direction within each group. For heavy-ball momentum one and step size one, the error in group j obeys

$$q_{j,t+1} = 2 \cos(\omega_j) q_{j,t} - q_{j,t-1}, \quad \lambda_j = 2(1 - \cos \omega_j).$$

The selected irrational frequencies produce r independent phases. All 32 original coordinates move. Three seeds vary bases, teacher directions, and probes, but the normalized training-loss dynamics are identical up to rounding for a fixed spectrum. These are coordinate/observer checks rather than three independent loss processes.

Table 7: Known-mode regression: median scalar MG. Settings are fixed across r . A longer record does not uniformly reduce bias.

r	Loss, 2,048	Loss, 8,192	Probe, 8,192
1	0.90	1.12	1.01
2	2.39	2.57	2.55
4	3.71	4.99	4.11
6	5.88	7.00	4.98

Selective damping sets momentum to 0.98 in two of four groups at step 8,192. The comparison uses disjoint 8,192-point windows before damping and after the transition, with the latter starting at 12,288. Across three seeds, loss-based MG changes from 4.97 to 2.56 in every run; the median probe estimate changes from 4.15 to 2.63. The corresponding loss values are deterministic up to the tested seed changes in coordinates and probes, so the table reports the three-run range rather than treating the repeated loss curves as independent trajectories. With ordinary momentum 0.99 on all groups, learning converges and late losses reach rounding scale. Standardization can then produce spurious high estimates; the absolute pre-standardization scale must be checked.

Table 8: Three-seed variation in the selective-damping regression experiment.

Signal	Phase	Median	Minimum	Maximum
loss	before	4.971432	4.971432	4.971432
loss	after	2.562791	2.562791	2.562791
probe error	before	4.153513	3.601192	4.206165
probe error	after	2.634185	1.995991	2.680959

C.3 Learned autonomous generators

The 1,000-unit generator evolves as

$$\begin{aligned} \mathbf{h}_{t+1} &= 0.9\mathbf{h}_t + 0.1(J \tanh \mathbf{h}_t + U\mathbf{z}_t), \\ \mathbf{z}_t &= A^\top \tanh \mathbf{h}_t. \end{aligned}$$

Here $\mathbf{h}_t$ is the network state, J the recurrent matrix with gain 1.2, U the output-feedback matrix, and A the readout trained by FORCE/RLS for 100,000 simulation steps. Frequencies are proportional to $1, \sqrt{2}, \sqrt{3}, \sqrt{5}$ for T1–T4; H2/H4 use integer harmonics, and M4 uses $1, 2, \sqrt{2}, 2\sqrt{2}$. All seven tasks use seeds 1–5. After training, we discard 5,000 burn-in steps and record 32,768 autonomous samples. The reported MG value is the median over four disjoint 8,192-point windows on neuron zero. Neurons one and two provide fixed robustness checks.

The Lyapunov reference uses twelve tangent vectors and a near-zero threshold of -0.02 chosen in advance. The number of near-zero exponents agrees with the target phase dimension in 31/35 cases. H2 has weakly contracting directions near the threshold and is overestimated in four seeds. Thus even this reference has finite-time ambiguity. MG has finite-resolution error as well: one periodic H2 record gives 0.73, T2 has median 2.84, and T3/T4 overlap. The reliable comparison is the distinction between phase structures, especially in the harmonic references; exact recovery across $d = 1$ –4 is not established.

As a robustness check, we re-evaluated the stored H4, M4, and T4 generators on three neurons, two windows, and three additive-noise levels. On neuron zero, MG preserved the expected $H4 < M4 < T4$ ordering on all

Table 9: CNN interventions at all tested strengths, on four held-out seeds. Changes are medians of paired record ratios. Update covariance and the fraction of moving parameters provide independent comparisons. Stronger pruning does not always produce a larger MG decline.

Intervention	MG before	MG after	MG change	Update PR change	Moving parameters
No intervention	4.31	4.32	+0.4%	+40.3%	100.00%
Batch $\times 4$	4.31	4.85	+12.8%	-7.3%	100.00%
Log $\times 10$	4.31	4.32	+0.4%	+40.3%	100.00%
Smoothed log	4.31	4.44	+3.3%	+40.3%	100.00%
Step / 3	4.31	4.35	+0.9%	-5.4%	100.00%
Step / 10	4.31	3.95	-8.3%	-21.6%	100.00%
Step / 100	4.31	3.85	-11.5%	-20.9%	100.00%
Freeze first two convolutions	4.31	4.27	-1.8%	-13.6%	65.31%
Train head only	4.31	3.51	-18.5%	-43.5%	2.25%
Train head bias only	4.31	3.58	-16.7%	-68.4%	0.07%
Prune 50%	4.31	3.80	-12.7%	+14.7%	50.30%
Prune 80%	4.31	3.59	-17.5%	-10.3%	20.48%
Prune 95%	4.31	3.79	-11.5%	-45.3%	5.55%

five seeds for 4,096 samples with no added noise and with noise of standard deviation 0.01 times the signal standard deviation. At noise level 0.05, the ordering held on two of five seeds for the shorter window and on none for the longer window. The corresponding median estimates were (1.14, 4.56, 6.30) in the clean short-window condition and (5.53, 5.12, 6.75) in the noisy short-window condition. This stress test supports robustness to small observation noise, while making the breakdown at larger noise explicit.

For the separate 256-unit motion experiment, gain is 1.5 and FORCE learns a figure-eight target for 40,000 simulation steps. Each frozen checkpoint is evaluated after 2,000 burn-in steps over 8,192 recorded states. The chaotic initializations comprise the first five eligible seeds among sequential candidates 6–30, selected by a positive pretraining Lyapunov criterion; thirteen candidates were examined, yielding seeds 7,9,15,16,18. No MG value enters selection. The unselected initial seeds 1–5 are retained separately because some start nonchaotic. This is conditional evidence for learning from chaos, not an estimate of how often random networks become chaotic or learn a stable cycle.

D APPLIED TRAINING PROCEDURES AND FULL COMPARISONS

D.1 CNN and ResNet interventions

The small CNN has three convolutional layers with 16, 32, and 32 ReLU channels, 3×3 kernels, max pooling after the first two layers, adaptive average pooling, and a ten-class linear head; it has 14,666 parameters and no normalization. It uses 10,000 CIFAR-10 images, SGD step 0.02, momentum 0.9, weight decay $5 \cdot 10^{-4}$, and batches of 64 sampled with replacement. The graded study runs 10,000 updates with an intervention at update 4,000. The before/after summaries use windows lying within the designated pre/post intervals, excluding the immediate mixed transition.

The learning-rate interventions divide the step size by 3, 10, or 100. Freezing leaves the later layers, only the head, or only its bias trainable. Pruning masks 50%, 80%, or 95% of weights. Some interventions rebuild SGD for the remaining trainable parameters, which also changes optimizer state. Their measured effect therefore includes that change and cannot be attributed to the number of parameters alone. Increased-batch references multiply batch size by four. Rescaling multiplies the base log by ten after update 4,000. Smoothing replaces that suffix by the causal 16-sample moving average $\bar{x}_t = \sum_{j=0}^{15} x_{t-j}/16$, using the preceding original samples at the boundary; neither transformation retrains the network.

The CNN IAAFT comparison uses two surrogates per window and reports their median MG, with RNG seed equal to the window start. Each surrogate uses 100 alternating spectrum/rank projections. Endpoint matching can trim up to 15% of the window before randomization, so spectrum and marginal preservation refer to the retained segment; it is a descriptive reference, not a matched-length significance test.

Detector development searches $M \in \{2, 3, 4\}$ and $B \in \{3, 4, 6\}$. For each pair, its drop threshold is the smallest

giving no false positives on development nulls, plus a margin of 0.02. The selected pair maximizes development event detection, with shorter delay as a tie breaker. Companion-statistic directions are chosen on development data as well. The rule is frozen for the new graded seeds and for ResNet. The selected MG threshold is 0.0882988401. The primary 27/28 result covers seven strong intervention types on four seeds; weak types are reported separately. The 16 references comprise base, increased batch, rescaled base, and smoothed base on the same four seeds. Thus these totals describe a matched benchmark rather than independent Bernoulli trials.

D.2 Natural scalar baselines

We computed additional scalar features on the same saved parameter-norm records. Permutation entropy measures the diversity of local ordinal patterns (Bandt and Pompe, 2002); sample entropy measures the recurrence of similar short templates (Richman and Moorman, 2000); normalized increments measure local motion; and spectral entropy summarizes the detrended periodogram. We also tested a one-sided Page CUSUM (Page, 1954) on each window feature. Every feature used the same event definition, development seeds, detector family, and held-out seeds as MG. These comparisons test complementarity, not population-level superiority.

Table 10: Natural scalar features on the held-out CNN benchmark. Each detector is fixed on the earlier seeds. Successful detections cover seven strong intervention types across four seeds; false positives cover four reference types across four seeds.

Feature	Hits	False positives	Median delay
MG	27/28	0/16	1,000
Abs. increments	27/28	0/16	500
Norm. increments	28/28	0/16	1,500
Perm. entropy	28/28	7/16	1,500
Sample entropy	28/28	4/16	1,500
Spectral entropy	4/28	2/16	2,750
Trend crossings	15/28	5/16	1,000
Lag-one corr.	0/28	0/16	—
Detrended std.	28/28	10/16	500
Level	4/28	0/16	1,000

Table 11: False positives by type for the block detector. The reference records share the four held-out seeds; the table therefore describes this matched benchmark rather than independent false-positive probabilities.

Feature	Base	Batch $\times 4$	Rescaled	Smoothed
MG	0	0	0	0
Abs. increments	0	0	0	0
Norm. increments	0	0	0	0
Perm. entropy	3	0	3	1
Sample entropy	1	0	1	2
Spectral entropy	0	2	0	0
Trend crossings	1	2	1	1
Lag-one corr.	0	0	0	0
Detrended std.	2	2	4	2
Level	0	0	0	0

The primary block detector uses $W = 1,000$, stride 500, and excludes feature windows ending at or before update 1,500 from the detection history. MG uses $M = 2$, $B = 3$, and a drop threshold of 0.08829884; each baseline selects its direction and threshold on the development records. Permutation entropy uses ordinal order five, unit delay, stable sorting for ties, and normalization by $\log(5!)$; sample entropy uses template length two, unit delay, Chebyshev distance, tolerance 0.2 times the window standard deviation, and self-match exclusion; normalized increments divide mean absolute increments by the window standard deviation; spectral entropy uses the normalized detrended periodogram; trend crossings count sign changes after linear detrending. Page CUSUM uses the same features, a past-only warm-up, drift selected from $\{0.25, 0.5, 1\}$, and the parameters in Table 12. The block score is $s(1 - \text{median}_M / \text{median}_B)$, with direction $s \in \{-1, 1\}$; MG fixes $s = 1$. Development searches $M \in \{2, 3, 4\}$, $B \in \{3, 4, 6\}$. Its threshold exceeds the largest reference score by 0.02, and selection maximizes event detections, then minimizes median delay. A baseline ratio is skipped when the reference magnitude is at most 10^{-12} .

CUSUM fixes its mean μ_0 and scale $\sigma_0 = \max(\text{std}_0, 0.01|\mu_0|, 10^{-10})$ from feature windows ending at 2,000–3,500. Starting at zero, $C_j = \max\{0, C_{j-1} + s(\mu_0 - v_j)/\sigma_0 - \kappa\}$; a detection occurs when $C_j > h$. Both directions and

Table 12: Selected detector parameters. The block sign is listed; all selected CUSUM signs are recorded in `new_results/rules.json`. Full-precision values in that file define the implemented thresholds.

Feature	M	B	Sign	δ	CUSUM drift	CUSUM h
MG	2	3	1	0.0882988	0.25	44.3109
Abs. increments	2	3	1	0.338305	1.0	19.7929
Norm. increments	4	4	-1	0.0289438	0.25	5.90329
Perm. entropy	2	6	-1	0.339038	1.0	7.79449
Sample entropy	2	3	-1	0.186699	0.25	9.86063
Spectral entropy	3	3	-1	1.29961	0.25	6.74364
Trend crossings	2	4	1	0.662857	0.5	2.31626
Lag-one corr.	2	3	1	0.0238814	0.25	0.653369
Detrended std.	2	3	-1	0.305439	0.25	0.5
Level	2	3	1	0.02	0.25	0.5

the stated drift grid are fixed, with h exceeding the largest development-reference score by 0.5. These rules are causal but warm-up must finish before scoring; they are not detectors available from the first training step.

Table 13: Past-only CUSUM on the held-out CNN records. Thresholds are selected on development records; the table reports strong-event detections, reference false positives, and median delay.

Feature	Detected	FP	Delay
MG	17/28	0/16	3,500
Abs. increments	27/28	12/16	2,500
Norm. increments	28/28	4/16	1,000
Perm. entropy	28/28	4/16	1,000
Sample entropy	28/28	4/16	1,250
Spectral entropy	5/28	1/16	500
Trend crossings	13/28	3/16	1,000
Lag-one corr.	1/28	0/16	500
Detrended std.	28/28	4/16	500
Level	0/28	0/16	—

On this benchmark, normalized increments detect 28/28 events with 0/16 reference false positives, compared with 27/28 and 0/16 for MG. Absolute increments match MG in detections and false positives with shorter median delay. Both entropy features detect all events but produce reference false positives. Simple features achieve higher detection rates in this CNN benchmark. MG provides a geometric statistic that is also evaluated on the specified generator, VAE, RL, and full-state timing benchmarks.

We evaluated ten additional seeds using a fixed procedure and historical detector rules fixed before training. This transfer test produced 19/30 intervention detections for MG; five of ten ordinary-training references contained threshold crossings, and six intervention branches had crossings before the assigned intervention time. Mean absolute increments produced 30/30 detections with no reference false positives. Normalized increments produced 29/30 detections and false positives in two of ten references. This confirmation indicates that the original CNN benchmark supports the fixed comparison above, but its detector threshold does not transfer universally across initialization and event schedules.

The MG horizon sensitivity on the original held-out records (Table 14) is 8/28, 16/28, 22/28, and 27/28 detections at horizons 500, 1,000, 2,000, and 5,000 updates, respectively; reference false positives remain 0/16 at all four horizons.

ResNet-18 runs use the full CIFAR-10 training set, new seeds 20–22, and scratch/pretrained settings. SGD uses momentum 0.9, weight decay $5 \cdot 10^{-4}$, initial step 0.02, and batch 128 without augmentation; the pretrained setting uses 64×64 images and ImageNet normalization. The pretrained network retains the standard 7×7 , stride-two stem and max pool and replaces its classifier by a ten-class head. The scratch network uses a 3×3 , stride-one stem, no initial max pool, and 32×32 CIFAR-normalized inputs. MG and the rule transfer unchanged. A 1,024-coordinate CountSketch of updates provides a tractable covariance reference. The head-only intervention succeeds in both settings; learning-rate and pruning detection does not. The six freezing records and 24 references reuse three seeds and two initial-model settings.

Table 14: Post-hoc horizon sensitivity with thresholds and first detections unchanged. Entries are detections within the specified number of updates; FP uses each complete reference record and is independent of the event horizon.

Feature	500	1,000	2,000	5,000	FP
MG	8/28	16/28	22/28	27/28	0/16
Abs. increments	15/28	16/28	26/28	27/28	0/16
Norm. increments	0/28	0/28	21/28	28/28	0/16
Perm. entropy	0/28	0/28	23/28	28/28	7/16
Sample entropy	8/28	13/28	27/28	28/28	4/16
Spectral entropy	0/28	0/28	0/28	4/28	2/16
Trend crossings	0/28	15/28	15/28	15/28	5/16
Lag-one corr.	0/28	0/28	0/28	0/28	0/16
Detrended std.	25/28	27/28	27/28	28/28	10/16
Level	0/28	4/28	4/28	4/28	0/16

D.3 ReLU-death stress test

A separate experiment uses six new seeds 30–35 and abrupt learning-rate shocks, with doses chosen on a pilot using activation death and accuracy, not MG. The tested learning-rate multipliers are 20, 30, 50, and 100 for one update; 15 for three updates; and 20 for two updates, followed by restoration of the base rate. A channel is classified as dead if it is zero on all 500 diagnostic images. An event requires an increase of at least 0.05 in the fraction of dead channels. There are 42 training runs, augmented by rescaled and smoothed base logs. Twenty-three runs meet the death criterion; two catastrophic whole-network failures are excluded from the prescribed eligible-event score and remain reported separately. The 42 runs comprise seven arms on six seeds: 23 meet the event criterion and 19 do not. Removing two catastrophic events leaves 21 eligible events; six rescaled and six smoothed base logs augment the 19 non-events to 31. Thus $42 - 2 + 12 = 52$ scored records, not 52 independent training runs.

Table 15: Boundary case: the transferred detector on ReLU death. Rules are fixed before the confirmation results.

Statistic	Detected / 21	False positives / 31
MG	12	12
Trend crossings	19	7
Loss stagnation	21	8
Detrended std.	21	25
Lag-one correlation	0	0

MG changes are nonmonotone in channel death: moderate loss of channels and catastrophic disruption can move the estimate in opposite directions. The CNN success therefore concerns the tested sustained training interventions and is not a generic failure detector.

D.4 Text VAE and preservation reference

We select 6,000 training and 256 fixed evaluation sentences with seed 20260929. Each has 8–24 words before the end-of-sentence token. A 2,000-token vocabulary is built from the training data. Shared embeddings have dimension 48, both GRUs have hidden size 64, and the diagonal Gaussian latent code has dimension 16. The decoder receives the code in its initial state and at every decoding step. Training uses teacher forcing, Adam with learning rate 0.001, batch size 32, gradient-norm clipping at five, and no dropout. The loss sums reconstruction NLL within each sentence, adds β times the KL term, and averages over sentences. Since the encoder observes the sentence, this reconstruction score differs from marginal generative likelihood and ordinary language-model perplexity.

The branches share weights, Adam state, and batch/noise generators at update 1,024. Before every optimizer update, we evaluate 16 fixed evaluation sentences with fixed Gaussian noise (seed 829). Early MG is the median of five windows ending at updates 512–1,024; late MG is the median of nine windows ending at 2,048–3,072. Both use stride 128.

Independent references are measured every 128 steps. The empirical-mixture mutual information (MI) estimate uses four fixed draws from each of 256 posteriors and is bounded above by $\log 256$ nats. The code-response score is the symmetric KL divergence between decoder token distributions with matched and cyclically shuffled posteriors, using matched noise and teacher forcing. We record information loss when both references fall below half their reference values, provided their early values were nontrivial.

The preservation branch uses free bits: at $\beta = 1$, it floors the batch-averaged KL of each latent coordinate at 0.5 nats before summing. We select this intervention on pilot seed zero using only the independent references. An earlier pilot with five extra encoder updates fails to preserve information and is retained as a negative result. Preservation requires both late references to retain at least half their early values and exceed the suppressed branch by at least a factor of two. All nine test seeds pass. This is a third continuation of each shared initialization, so the comparisons remain paired.

Table 16: Text VAE: late medians across nine confirmation seeds. MI and code response are independent of the MG calculation.

Branch	MI	Code response	MG
Low regularization	5.545	2.087	8.129
Suppressed code	0.327	0.021	4.172
Free bits	3.934	0.599	5.491

The suppression result is consistent across all nine seeds at lag one for window lengths 256, 512, and 1,024. At lag four, seven of nine seeds agree. The 1,024-point configuration has only one early window. Using ordinary mini-batch loss instead of the fixed probe gives the expected direction on only four seeds. Standard deviation and spectral entropy of the fixed probe decline on all nine. These comparisons show that a stable probe is important for detecting the information change in this experiment.

D.5 Cyclic-VAE monitoring

Fresh seeds 100–109 use the same architecture/data. We select thresholds on seed 100 and evaluate on seeds 101–109. After 1,024 steps at $\beta = 0.01$, three 2,048-step cycles linearly raise β from zero to one in their first halves, hold it at one in their second halves, and reset. Total length is 7,168 updates. Training is independent of monitor requests. References every 64 steps define loss when both normalized quantities are at most 0.5, and recovery when both are at least 0.65, each on two consecutive checks. The event time is the second check. There are 41 events with a complete 512-step detection horizon and two late events scored separately.

The monitor requests a reference when the absolute log change of its feature from the previous request exceeds its pilot-selected threshold, respecting a 128-step refractory period and a budget of 6, 12, or 24 extra checks. Every method also receives one initial check. For each budget, thresholds are selected on the pilot from 0.025, 0.05, 0.075, 0.1, 0.15, 0.2, 0.3, 0.4, 0.5, 0.7, 1, 1.5, 2, and 3. Features are evaluated every 64 updates in this cyclic experiment; the earlier preservation experiment uses stride 128. Decisions use no future values. Periodic checks use evenly spaced grid points with no favorable phase selection.

Table 17: Cyclic VAE: detected events out of 41 at all tested budgets. Primary budget is 12.

Method	$B = 6$	$B = 12$	$B = 24$
MG	18	28	36
Standard deviation	13	20	38
Spectral entropy	17	28	35
Training KL	8	13	31
Schedule	18	18	26
Periodic	12	38	40

At budget 12, MG makes 9.44 requests on average. Matching a periodic schedule to the number of requests for each seed yields 34 detections, compared with 28 for MG; this post-hoc comparison uses request totals but no event times. Against the main periodic comparator, mean per-seed recall for MG is lower by 24.4 percentage points, with bootstrap interval $[-36.7, -10.0]$. At budget six, MG detects 18 events versus 12 for periodic checks,

although a rule based on the known training schedule also detects 18. MG is therefore competitive at the smaller budget but does not improve monitoring overall in this test.

E ADDITIONAL LEARNING SETTINGS

MNIST. A 15,018-parameter tanh MLP is trained on a fixed balanced 2,040-image subset after downsampling to 14×14 . SGD uses momentum 0.9, batch 64, and no weight decay. At step 2,048 of 4,096, one paired branch reduces the learning rate from 0.1 to 0.01. A fixed 100-image loss probe is selected on the pilot. Detrended covariance rank declines more than the reference on all five new seeds; MG does so on four of five, with median contrast only 4.8%. The larger-window sensitivity is more favorable but does not replace the primary result.

Grokking. The four positive sketch runs comprise modular addition $y = (a + b) \bmod 113$ on seeds 42–44 and permutation composition in S_5 on seed 42. Training fractions are 0.30 and 0.50, respectively; zero-decay reference runs use the corresponding seed-42 tasks. A one-layer transformer has width 128, four 32-dimensional heads, MLP width 512, no layer normalization, and float64 arithmetic. AdamW uses learning rate 10^{-3} , betas (0.9, 0.98), batch 256, and decay 1.0 (addition) or 0.2 (S_5). Training lasts 20,000 or 15,000 loop iterations. The historical S_5 implementation performs two optimizer updates per loop iteration; this difference is retained when reporting its times. The unregularized full-batch comparison uses a width-500 quadratic perceptron at modulus 97, a 50% training split, and plain gradient descent on mean squared error. Random projections of parameter and logit trajectories support detrended covariance analysis. A sharp collapse and later recovery aligns with generalization in four positive regularized runs, with two references; a memorizing zero-weight-decay run can show simplification elsewhere. The short-window parameter-norm analysis uses 60 logged samples at the centers of the trajectory windows. Its grid crosses $E \in \{4, 6, 10\}$, $\tau \in \{1, 2, 4\}$, $k \in \{5, 20\}$ and embedding/ACF exclusion. The headline cell maximizes E subject to $(E - 1)\tau \leq 60/4$, retaining the frozen k and exclusion rule, and breaking ties by smaller τ : $E = 10$, $\tau = 1$, $k = 20$, embedding-span exclusion. This rule does not select the largest observed effect. The scalar decline is a change statistic at that resolution. Its 24/26 usable cells vary analysis settings on the same runs. The signal is not a unique indicator of generalization, and scalar roughness is an alternative explanation in this setting. We retain this as a complementary example rather than the sole applied validation. Linear detrending removes the scalar separation in the matched-window analysis. Covariance PR can change with drift, correlation time, or coherent motion even without a change in active dimension; random-walk and Ornstein–Uhlenbeck trajectory PCA illustrate this distinction (Antognini and Sohl-Dickstein, 2018). We have not established that the grokking effects persist against nulls matched jointly in temporal correlations and nonstationary drift. The observations therefore identify a change in trajectory and scalar statistics, not its unique dimensional cause.

Walker2d. PPO policies command six actuators in Walker2d-v5. Fixed-policy evaluations use a 512-step warm-up and 4,096 recorded steps. In phase-conditioned variants, the observation is augmented by sine/cosine phase and the reward is changed to encourage an intended periodic behavior. Subsequent comparisons use a common initial locomotion checkpoint with separate fine-tuning seeds, not independently trained starting skills. Scalar joint and action logs are analyzed alongside full-state return errors, inter-cycle variation, and responses to injected perturbations.

The smoothing objective is

$$L = L_{\text{PPO}} + \frac{\lambda}{6} \mathbb{E} \|\bar{a}_\theta(s_{t+1}) - \bar{a}_\theta(s_t)\|_2^2,$$

where $\bar{a}_\theta(s)$ is the policy mean clipped to $[-1, 1]$ per actuator. Episode boundaries are excluded. The environment reward and executed commands receive no extra smoothing. Separate tracking variants change the reward to $r'_t = r_t - \lambda(1 - \exp[-e_t^2/(2\sigma^2)])$, where e_t measures distance to the recorded reference cycle, either at its nearest point or at the prescribed phase. The reference has 152 samples; the external phase period is approximately 152.215 environment steps. The initial $\sigma^2 = 0.116779$ saturated; a separate development run gave 38.477 for the broader tracking experiment.

Actor and critic use separate 64–64 tanh networks, eight environments, 512-step rollouts per environment, batches of 256, five PPO epochs, learning rate decreasing from $3 \cdot 10^{-5}$ to $3 \cdot 10^{-6}$, clip 0.1, target KL 0.01, discount 0.99, GAE 0.95, entropy coefficient zero, value coefficient 0.5, and gradient clipping 0.5. Normalizations are frozen. Fine-tuning branches use a common initial checkpoint after 655,360 transitions, then 1,048,576 additional transitions per branch; their seeds do not represent independently learned initial policies.

Moderate action-smoothing penalties give the most reproducible scalar result: two five-seed series show lower aggregate action-log MG. In the final five-seed comparison, penalty strengths 0.25 and 1 give median MG/reference ratios 0.801 and 0.604, with a decline on every seed. Normalized increments also decline in four and five seeds and are cheaper. In the stricter full-gait experiment, only two of five pairs meet the joint improvement criterion, and only one pair has lower MG under every prescribed reconstruction. Thus a simpler command signal is not sufficient evidence of a simpler or more stable whole-body gait. Timing advantages below concern repeated perturbation probes, not these elementary scalar competitors.

E.1 Selecting policies for actuator shifts

We evaluated historical PPO policies on new nominal and deployment episodes. The 16 candidates per fine-tuning seed cross smoothing strengths 0, 0.25, 1, 4 with checkpoints at 262,144, 524,288, 786,432 and 1,048,576 transitions. Development seed 231 is excluded from the summary; seeds 232–235 provide the four confirmation units. All policies share one initial learned skill. Earlier results informed the setting, so this is new-outcome evaluation on existing policies, not an independent sample of newly learned skills.

Each selector receives three nominal episodes (reset seeds 83001–83003), with 256 warm-up and 2,048 measured environment steps per episode. Eligible candidates complete at least two episodes and have mean zero-padded return at least 90% of the largest nominal mean. MG selects the lowest median estimate from the scalar action norm over complete episodes, using $E = 20$, $\tau = 8$, $k = 20$, $T = 312$, and dither seed 123. Entropy, normalized squared increments, and minimum scalar return error over lags 20–250 use the same log. Action smoothness J_1 uses all six commands. Reward selection maximizes the mean over all episodes, including falls. Every method has the same eligibility gate. The fixed pilot choice is smoothing strength 0.25 at the final checkpoint, falling back to nominal reward if ineligible.

Deployment applies $v_t = (1 - \alpha)u_t + \alpha v_{t-1} + \epsilon_t$, with $v_{-1} = 0$, commanded action u_t , and independent Gaussian components of ϵ_t with standard deviation σ . The environment receives v_t clipped to $[-1, 1]$; the recursion retains the unclipped value. The six conditions cross $\alpha \in \{0.05, 0.1, 0.2\}$ and $\sigma \in \{0, 0.02\}$ and apply also during warm-up. Five new reset seeds 84001–84005 per condition give 30 target episodes per candidate. Return is the measured reward sum divided by 2,048, with a zero remainder after falls. Selectors never use confirmation target outcomes; the oracle does. The sampling interval is 0.008 seconds.

Table 18: Policy selection under hidden actuator shifts: mean over four fine-tuning seeds. Completion is over all 120 selected target episodes. Random denotes expected uniform choice among eligible policies; the oracle uses target returns.

Selector	Return	Completion
MG	4.313	90.8%
Nominal reward	4.005	80.0%
Spectral entropy	4.480	97.5%
Scalar recurrence	4.398	95.0%
Action smoothness	2.999	47.5%
Norm. increments	3.064	49.2%
Fixed pilot	4.310	90.8%
Random eligible	4.286	90.9%
Oracle (eligible)	4.671	100.0%

MG exceeds nominal-reward selection on three of four seeds, with paired mean difference 0.308 (7.7%); a two-sided sign-flip test gives $p = 0.375$. It does not outperform spectral entropy, and its mean advantage over the fixed pilot is only 0.0025. Reset repetitions and candidate checkpoints are correlated and are not independent statistical units. MG-based selection therefore provides a descriptive benefit over one common rule, not established superiority over competing selectors.

On an Intel i5-12500H CPU with one numerical thread, a 2,048-point MG calculation takes 0.516 seconds, compared with 0.00014 for entropy and 0.0062 for scalar recurrence. Three nominal episodes take 5.52 seconds, while 30 target episodes take 44.7 seconds on the benchmark policy. Nominal acquisition plus three MG calculations is approximately 7.06 seconds; these component medians do not establish a universal speed advantage, and entropy uses the same acquisition budget. Training all candidate policies is a shared sunk cost. Choosing an earlier stored

checkpoint does not demonstrate causal early stopping or curriculum improvement.

An earlier stress test selected among four final policies using delay-free observations and applied delays of 1–3 entire environment steps after warm-up. On five seeds, MG and the pilot-selected fixed strength 1 choose the same policies, with return 0.541 versus 0.319 for nominal reward, but only 2/150 target episodes finish without falling. This severe-shift test is retained as a negative deployment result.

F TIMING PROCEDURES AND INFORMATION REQUIREMENTS

Timing separates simulation/training, recording an observable, analysis of an existing record, and extra diagnostic interventions. The comparisons are local implementation benchmarks with the listed configurations. They do not establish a universal complexity ordering among all algorithms for the same scientific question.

F.1 Computing infrastructure and provenance

The FORCE timing report identifies an Intel Core i5-12500H CPU. Its saved environment specifies Windows 10 (build 18363), Python 3.13.12, NumPy 2.4.3, SciPy 1.17.1, and one BLAS thread; the benchmark uses float64 arithmetic and three warmed sequential repetitions. No GPU is used in this benchmark.

The cyclic-VAE benchmark records CPU execution on Windows 10 (build 18363), Python 3.13.12, and PyTorch 2.10.0+cpu, with two PyTorch threads, one estimator thread, and forty interleaved repetitions. Its metadata do not identify the CPU model. The Walker timing metadata record the same operating system and Python version, an Intel64 Family 6 Model 154 Stepping 3 processor, one thread, and five repetitions.

For the new ten-seed CNN confirmation, the local machine identifies an Intel Core i5-12500H (12 physical cores, 16 logical processors); the training script uses CPU execution with six PyTorch threads. This machine identification was collected during manuscript revision on October 2, 2026 and does not retrospectively establish the hardware of every earlier experiment. The historical CNN timing output does not record a CPU model. Absolute runtimes and speed ratios therefore describe the documented local implementations; a complete hardware inventory for all historical runs is unavailable.

F.2 Benchmark-specific timing and storage

CNN. The saved graded result reports median MG processing of 9.49 ms per 1,000-point window. A float32 scalar record of 10,000 updates takes 39.06 KiB; the corresponding 14,666-dimensional trajectory takes 559.46 MiB. Parameter-norm acquisition is an $O(P)$ reduction, not zero-cost logging. CountSketch update references in ResNet use 1,024 stored coordinates per update and require computing the sketch. A streaming reference would change the storage comparison.

FORCE. The 256-unit endpoint benchmark replays fixed-checkpoint autonomous trajectories. Median MG times before/after training are 0.835/0.598 s; full-spectrum times are 12.006/11.551 s. With both $E = 20$ and $E = 40$ calculations, scalar analysis takes 2.233/1.213 s. Scalar recording costs 0.102/0.166 s, including rollout; this is separate from analysis. The largest exponent costs 0.198/0.153 s and elementary scalar baselines 0.0009/0.0015 s. The full spectrum supplies information that a single exponent or scalar score does not, and MG does not recover the exponents themselves.

In the 1,000-unit generator, analysis of four 8,192-point windows takes 8–23 s across learned-task medians. The twelve-vector Lyapunov calculation takes 41–66 s; full-state PCA is about 0.2 s. The advantage over PCA is its input requirement, not time. The difference from the millisecond analysis of CNN windows is material and follows from record/configuration and implementation costs.

Coupled oscillators. A separate benchmark evaluates dense phase networks of 32,64,96,128 oscillators at two coupling levels using a shared saved 1,536-point trajectory for each comparison. Scalar analysis computes MG and LB together; the full spectrum integrates all tangent directions with QR steps. Medians use five scalar and three spectrum repetitions. At size 128, scalar times are 0.061/0.049 s and full-spectrum times 1.309/1.124 s. Scalar storage is 128 times smaller. At size 32, the full spectrum is faster. An additional same-seed 64-unit comparison with 4,096-point scalar windows also favors the spectrum (roughly 1.0–1.4 s versus 1.6–2.2 s), so the short-window

speedup is not universal. The same-seed coupling sweep shows a collective contraction of the spectrum and MG approaching one after synchronization; finite-time exponent thresholds are not exact dimension labels. This is a dynamical-system benchmark, not neural-network training.

VAE. The cyclic benchmark uses one warmed process, two PyTorch threads, one estimator thread, and forty interleaved repetitions. Median costs are 7.134 ms per fixed probe, 12.890 ms per MG window, and 209.323 ms per combined reference. Cost totals multiply component medians by the actual numbers of operations, including 7,168 probe acquisitions and 105 feature calculations. They are estimates of deployment cost, not separately timed complete monitoring runs. Other jobs may share the machine. At budget 12, acquisition alone costs 51.14 s, leading to the 54.68 s MG total. Cheap statistics using the same probe incur the same acquisition cost. Periodic checks require no such probe stream.

Walker2d. In two fixed-horizon comparisons, MG of an existing scalar is approximately 21–25 times cheaper than repeated perturbed simulations; including the doubled-embedding check reduces this to about 10–11 times. These probes answer questions about stability that MG alone cannot answer. On the final scalar benchmark, MG costs 369 ms per window, versus 0.103 ms for spectral entropy, 3.782 ms for a scalar return statistic, and 0.048 ms for normalized increments. The data-saving property is shared by all four scalar methods.

G FORECAST SELECTION AND SAMPLED-QUERY ESTIMATION

Training and observable. This experiment uses new 256-unit networks with the FORCE/RLS dynamics described in Appendix C. The seven task families are T1–T4, H2, H4, and M4. Each network trains for 40,000 updates with recurrent gain 1.2, integration step 0.1 and RLS updates every two steps; RLS starts at the identity. Target frequencies and random phases follow the generator construction above. An eighth, untrained reference has gain 1.5; it is not assumed to be chaotic and can settle to a fixed point. At frozen weights, each run has 5,000 burn-in steps and 8,192 recorded samples. The scalar is $\tanh(h_{t,2})$, using zero-based neuron index two. We use observations 4,096–6,143 as the prefix and 6,144–6,175 as the future target. No future values enter normalization, feature extraction, candidate fitting, or selection.

All trained networks are included, even when they fail to reproduce the higher-phase targets. Forecast errors measure their actual activity; the intended phase dimension is not assumed to have been learned.

Eight candidate forecasts. The pool comprises the last value, prefix mean, repetition at lag 314, ridge regression with 8 or 48 lags, five-nearest-neighbor regression with 8 or 48 lags, and ridge regression on 64 random Fourier features of a 48-lag input. Forecasts are direct 32-output regressions, fitted on all complete input/target pairs inside the prefix. Ridge uses penalty one and intercept penalty 10^{-8} on standardized inputs. The Fourier feature map has bandwidth $1/48$ and random seed 42. Nearest neighbors have uniform weights. Learned predictions are clipped at ± 10 prefix standard deviations. NMSE is squared forecast error averaged over the 32 future values and divided by prefix variance; no square root is taken. Prefix standard deviation below 10^{-10} triggers a shared persistence fallback, scored with variance floor 10^{-12} for every selector.

Selector training and search history. Each selector predicts the eight candidate losses and picks their minimum. Training uses seeds 601–618 (144 task records); three-fold cross-validation groups by seed. Four candidate selector architectures are considered equally for every feature set: decision trees of depth 2/3 with minimum leaf size eight, and 100-tree random forests of depth 3/6 with minimum leaf size five. Random seed is 42. Missing-value imputation and indicators are fit inside the training folds. The depth-two MG selector generally chooses nearest-neighbor prediction for cyclic records and ridge regression for more complex records. The task label and target phase dimension are never selector inputs.

An exploratory search over generators, neurons, horizons and sensor datasets preceded selection of the primary observable and forecast horizon. Seeds 619–630 identified the short-horizon candidate; seeds 631–650 tested it, after which stronger geometric competitors were added. All full-MG and geometric selector rules were then frozen before seeds 651–680 (30 new groups, 240 records). The sampled-query variant was proposed after measuring the cost of full MG and tested with the same frozen selectors on seeds 681–700 (20 further groups, 160 records). We report the two sets of test seeds separately. Resampling groups the eight tasks sharing each seed. The results concern this network family and task mixture; the bundled records document the search history.

Competitors and information access. All feature-based competitors use the same development runs and selector search. Cheap features are spectral entropy, normalized squared increments, lag-one correlation, minimum scalar recurrence error over lags 6–288, spectral peak concentration, linear trend, standard deviation, and mean. The geometric audit includes TwoNN and original Levina–Bickel pooling on the same delay cloud, its covariance participation ratio, permutation entropy of order five, and sample entropy with template length two and tolerance 0.2 standard deviations in Chebyshev distance. Held-out model search fits all eight candidates on four rolling prefixes with 32-point validation targets covering the last 128 known observations, then refits the chosen candidate on the entire prefix. Learned selectors receiving these validation errors have additional input and fitting cost. “All except MG” combines the cheap features, validation errors, and non-MG geometric features; “all including MG” adds MG to that same set. The oracle uses future errors and is only a lower bound. Table 19 includes these stronger alternatives.

Table 19: Mean NMSE for all forecast selectors. Columns use distinct sets of test seeds; lower is better. Earlier MG was fit during the initial generator screen. The second set supports the query-budget comparison.

Selector	30 seeds	20 new seeds
MG	0.03138	0.03637
Earlier MG rule	0.03219	—
Best fixed on development runs	0.04475	0.06041
Held-out model search	0.03224	0.04074
Cheap features	0.04517	0.05923
Cheap + held-out search	0.03617	0.03379
Held-out errors	0.03997	0.03424
Held-out errors + MG	0.03423	0.03167
Cheap + MG	0.03138	0.03637
Cheap + held-out search + MG	0.03534	0.03994
TwoNN	0.03540	0.04364
Levina–Bickel	0.04054	0.04732
Delay covariance PR	0.04397	0.06098
Permutation entropy	0.04484	0.05384
Sample entropy	0.04718	0.05954
All features except MG	0.03669	0.03939
All features including MG	0.03420	0.04071
Oracle (future targets)	0.00759	0.00847

On seeds 651–680, MG gives NMSE 0.03138 versus 0.03224 for ordinary held-out search; seed-bootstrap uncertainty includes no difference. On seeds 681–700, MG gives 0.03637 versus 0.04074: a 10.7% lower sample mean, but the paired improvement interval is $[-0.00729, 0.01758]$ over 20 seed groups (20,000 bootstrap resamples, random seed 811). MG wins on nine of those 20 groups. Cheap features plus learned held-out-error selection achieve 0.03379 and held-out-error features plus MG achieve 0.03167. Restricting seeds 651–680 to trained networks gives MG 0.03542 versus 0.03943 for TwoNN. The uncertainty against the strongest selectors remains.

Sampled-query MG. Use $E = 20$, $k = 20$, the adaptive lag of Appendix B, an autocorrelation Theiler rule capped at 150, and dither seed 123. Standardization, floors, and temporal exclusion match the original kernel. For n delay vectors, select $q = \min(128, n)$ distinct query indices uniformly across the record, with zero-based indices $\lfloor j(n-1)/(q-1) \rfloor$. All n vectors remain candidate neighbors. With original time indices retained in the exclusion, compute

$$\hat{d}_{\text{MG}}^{(q)} = \frac{q(k-1) - 1}{\sum_{i \in Q} \sum_{j=1}^{k-1} \log(r_{ik}/r_{ij})}. \quad (5)$$

The approximation equals full MG when Q contains every query. Numerical-floor checks use the queried distances and sums. Insufficient neighbors, nonfinite values or constant records yield unavailable estimates. The unchanged selector uses imputed scores to choose a forecast.

Distance computation uses blocks of 64 queries, followed by temporal exclusion, selection of k admissible neighbors and pooling of log ratios. Its cost is $O(qnE)$ with bounded block memory. Both versions use the full neighbor cloud; comparison with the original KDTree therefore measures both query reduction and implementation changes.

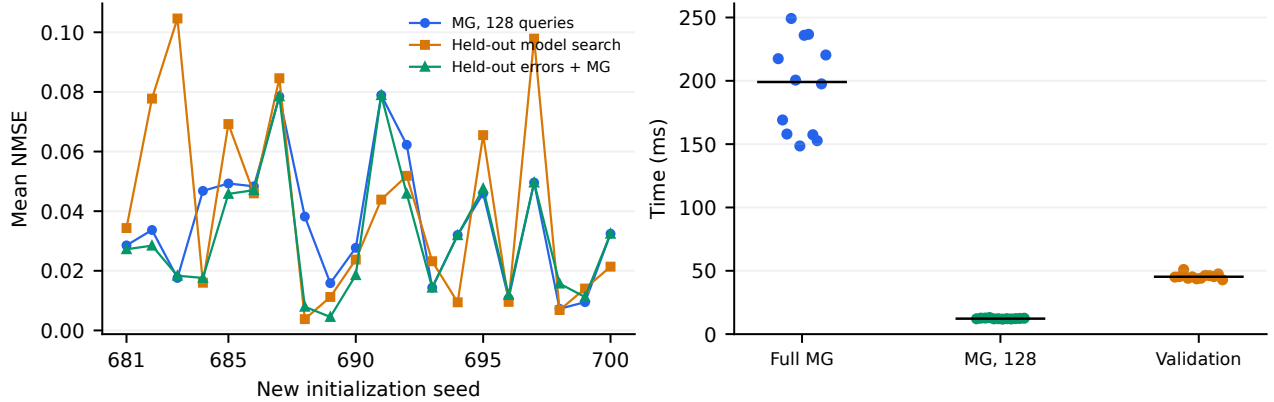


Figure 7: Forecast selection: quality and cost. Left: each new seed group averaged over eight tasks; lower NMSE is better. Right: medians over seven timing repetitions for each of 12 matched inputs; black bars are medians over inputs.

On all 160 records from seeds 681–700, $q = 128$ retains exactly the full-MG model choice and therefore the same forecast error. It does not produce an identical numerical estimate in general. The sensitivity budgets $q = 64$ and 256 also preserve those choices on these runs; 128 is the primary budget, not a test-selected optimum. Twelve audited records check exact full-query agreement, tree/block neighbor-distance agreement, and affine rescaling; constant and nonfinite inputs are rejected. Forecast-choice agreement alone does not certify absolute active dimension.

Paired runtime procedure. Serial measurements use an Intel Core i5-12500H CPU, one numerical thread, seeds 681, 690, and 700 crossed with tasks T1, T2, H4, and T4, seven interleaved repetitions per input after warm-up. Medians per input are formed before aggregating the 12 inputs. Overall median times are 199.1 ms for full MG selection, 12.2 ms for $q = 128$, and 45.3 ms for held-out model search. Median paired speed ratios are 16.3 and 3.7, respectively. Each includes the feature, selector, and final forecast fit. The held-out search performs 32 validation fits and one final fit; MG fits only its selected candidate. Disk access, acquisition of the common prefix, generation of the development bank, and offline selector training are excluded. These timings concern repeated selection from available records; elementary scalar statistics can be faster.

H COMPARING INTERVENTION RULES

We compare MG with scalar baselines and fixed schedules for choosing when to stop training, freeze layers or prune units. All rules use the same development runs and candidate intervention times.

MNIST images are average-pooled to 14×14 . A fixed balanced split uses 2,000 training images, 500 disjoint clean held-out evaluation images from the original training pool, and 2,000 official test images (split seed 1006). The network is a 196–128–128–10 ReLU MLP trained by Adam at 10^{-3} , batch 64, for 2,048 updates. For stopping experiments, exactly 40% or 60% of training labels are replaced by a randomly selected wrong class. Each initialization seed has its own corruption mask. Observers are parameter norm and loss on a fixed random subset of 128 training examples with their possibly corrupted labels, not clean probe labels. The reported experiment replaces the earlier single-class probe pilot.

Candidates act at updates 256, 512, 768, 1,024, or 1,536; no threshold crossing means continuation to 2,048. Trailing windows have 256 samples, $E = 10$, $\tau = 1$, $k = 10$, and $T = 19$. MG competes with mean level, normalized trend, detrended standard deviation, increments, entropy, and lag-one correlation. All select one of the two observers and threshold direction/level with the same pilot search. Thresholds maximize pilot clean-validation accuracy minus 0.02 times the arithmetic budget. We choose rules on seeds 0–2 and evaluate them on seeds 10–17. The evaluation trajectories predate the saved rule file, but rule fitting reads only pilot records; this was not prospective preregistration. A separate clean-validation checkpoint choice is an offline comparator, not a causal oracle for test accuracy. The no-intervention fallback uses the final recorded accuracy.

On clean training, freezing disables gradients through both hidden layers and continues fitting the final head. Pruning physically reduces both hidden layers to width 64, retaining units according to incoming-row norm times outgoing-column norm, copying retained weights/biases and sliced Adam moments. Each selected action time is executed from its saved checkpoint with the same minibatch RNG, then trained to update 2,048. Post-action outcomes are actual continuations, not inferred from the unchanged base path.

Table 20: Eight-seed intervention results, including strong simple comparisons. MAC ratio is an approximate matrix-arithmetic budget relative to uninterrupted dense training; it excludes logging, monitoring, nonlinearities, optimizer work, and memory overhead and is not a wall-time speedup.

Task	Rule	Accuracy (%)	MAC ratio
Stop, 40% noise	MG	84.75	0.219
Stop, 40% noise	fixed	84.72	0.250
Stop, 40% noise	level	84.90	0.188
Stop, 40% noise	entropy	84.67	0.203
Stop, 40% noise	never	64.11	1.000
Stop, 60% noise	MG	76.58	0.125
Stop, 60% noise	fixed	76.58	0.125
Stop, 60% noise	level	76.58	0.125
Stop, 60% noise	entropy	76.58	0.125
Stop, 60% noise	never	43.83	1.000
Freeze, clean	MG	91.53	0.535
Freeze, clean	fixed	91.32	0.515
Freeze, clean	level	91.32	0.515
Freeze, clean	entropy	91.32	0.515
Freeze, clean	never	92.29	1.000
Prune, clean	MG	92.44	0.572
Prune, clean	fixed	92.52	0.628
Prune, clean	level	92.50	0.618
Prune, clean	entropy	92.38	0.553
Prune, clean	never	92.29	1.000

At 40% noisy labels, MG stopping yields 84.75% test accuracy at 448 updates on average, versus 64.11% at the full budget; the mean-level rule yields 84.90% at 384 updates. At 60%, MG and fixed early stopping choose update 256 and both yield 76.58%, versus 43.83% for the full budget. For clean-data pruning, MG yields 92.44% accuracy at 57.2% of the dense arithmetic budget, versus 92.29% for uninterrupted dense training. Fixed-time pruning yields 92.52% at 62.8%, while entropy gives 92.38% at 55.3%. The interventions help, but simpler rules can match them. Rule selection uses clean labeled pilot data and depends on the noise setting. Application to later runs avoids those labels; logging and monitoring remain extra costs.

I FURTHER TESTS OF PRACTICAL DECISIONS

We compare MG with scalar and task-specific baselines in six learning settings (Table 21). Each study separates rule selection from evaluation. The settings address distinct decisions; their metrics do not measure a common dimension.

I.1 Self-supervised representation selection

Task and architecture. We train an MNIST encoder with SimSiam, BYOL, VICReg or SimCLR objectives. The encoder is a 784–256–256 MLP with batch normalization and ReLU after each linear layer. Its representation is $h \in \mathbb{R}^{256}$. A 256–256–64 projector produces z ; one configuration instead uses 16 output coordinates. SimSiam/BYOL usually use a predictor with hidden width 64. The 29 configurations vary step size, weight decay, augmentation strength, predictor, stop-gradient, projector normalization, loss coefficients, or contrastive temperature; the exact grid is bundled in `ssl/configs.py`.

Training uses 3,000 SGD steps, batch 128, momentum 0.9, and 100 warm-up steps followed by a constant learning rate. Two independently augmented images enter each update. The augmentation families combine affine shifts, rotations and scaling, erasure, pixel dropout, and Gaussian noise; their numerical ranges are in `ssl/ssl_core.py`.

Table 21: Scope of the additional studies. Values are from the final saved comparisons. Each row uses its own metric and evaluation population.

Decision	MG result	Relevant comparison and scope
SSL configuration selection	Mean Spearman 0.712; top-choice regret 0.41 pp	Embedding spread 0.730; encoder RankMe 0.321; five seeds, 29 configurations.
Transformer spike warning	22/23 events; AUROC 0.949	Roughness 23/23, AUROC 0.960 at the same false-positive rate; four test seeds.
Resetting after plasticity loss	Online accuracy 0.873/0.834 in two task families	Dormant-neuron rule 0.880/0.843; never resetting 0.845/0.776; 12 streams per family.
Learning-rate reduction	Final accuracy 0.574	Fixed late reduction 0.590; constant rate 0.543; 24 evaluated runs.
Population-size proxy in games	Correct cycle category 0.523	Harmonic count 0.713; state PCA about 0.99; 216 new games.
Transfer of one detector	26/87 events; 10/58 false positives	Detrended spread 85/87; 9/58 false positives across ten target settings.

These are small-model implementations of the published objectives (Chen and He, 2021; Grill et al., 2020; Bardes et al., 2022; Chen et al., 2020), rather than reproductions of their original large-scale benchmarks.

Independent evaluation and information access. A logistic-regression probe fits standardized h from 5,000 labeled training images with $C = 0.1$ and at most 300 solver iterations. It is scored on 5,000 official test images. Fixed subsets use seed 1234. Their labels never enter SSL training. Probe accuracy supplies the target for choosing the monitor’s log and direction on development seeds 0–1, then for scoring frozen selectors on test seeds 10–12. Seeds 20–21 add 58 further training runs after the initial comparison. Thus deployment of the selector is label-free, but its development is label-informed. The same probe images are reused across seeds, so this is a test of run variation on one task, not independent dataset generalization. The pilot seed 99 is excluded.

Each scalar selector chooses one of five recorded logs (loss, parameter norm, gradient norm, and representation/projector norms of one fixed image) and its sign using mean within-seed Spearman correlation on the development runs. No test outcome changes that choice. Primary MG uses $W = 1,000$, $E = 20$, $\tau = 1$, $k = 20$, and Theiler exclusion 19 on the final window. Its selected log is gradient norm and its sign is positive: higher MG predicts better representations in this task. This result concerns cross-run ranking, not an observed within-run dimension decrease. Undefined scores are ranked worst; no run is removed on the basis of downstream accuracy.

Representation proxies and scalar comparisons. RankMe (Garrido et al., 2023) is computed separately on h and z from 2,048 images. The α -ReQ comparison (Agrawal et al., 2022) uses a log–log slope fitted to the first 128 covariance eigenvalues of h , with both slope and distance-to-one variants. It does not reproduce the full search for a task-dependent preferred range in the original method. LiDAR (Thilak et al., 2024) uses 256 images with eight augmentations each, evaluated on z , with within-image covariance ridge 10^{-4} . Embedding spread is the mean coordinate standard deviation of unit-normalized z , multiplied by the square root of its dimension. Batch normalization can preserve a high encoder rank. Layer and augmentation choices also affect proxy quality, limiting this comparison to the specified implementations and grid.

All scalar methods receive the same candidate logs and development seeds. Competitors include roughness, recurrence error, spectral and permutation entropy, correlation dimension, TwoNN, delay covariance participation ratio, trend crossings, lag-one correlation, detrended spread, and level/change features. Table 24 includes the complete frozen comparison.

Results and uncertainty. MG has mean top-choice regret 0.408 percentage points versus 5.467 for random selection. Its ranking correlation falls from 0.755 on seeds 10–12 to 0.649 on seeds 20–21; embedding spread gives 0.729 and 0.732. Table 22 reports the pooled means.

The archived pairwise intervals resample the 29 configurations with replacement, sharing each draw across all five seeds (1,000 replicates, RNG seed zero). They are pointwise intervals conditional on these seeds, with no correction for comparing many methods. They exclude zero against encoder RankMe, the slope version of α -ReQ, LiDAR, and recurrence error; they include zero against projector RankMe, embedding spread, roughness, and spectral entropy. Table 23 preserves these intervals without interpreting them as seed-level significance.

Table 22: SSL configuration ranking. Spearman correlations are computed across 29 configurations within each seed and then averaged. Regret is the gap between the chosen and best probe accuracy, averaged over five seeds and expressed in percentage points. The two confirmation seeds are shown separately.

Score	Seeds 10–12	Seeds 20–21	All five	Regret (pp)
MG (gradient norm)	0.755	0.649	0.712	0.41
Embedding spread	0.729	0.732	0.730	1.46
Spectral entropy	0.653	0.507	0.595	1.00
Roughness	0.588	0.616	0.599	1.20
Delay covariance PR	0.652	0.587	0.626	1.08
Correlation dimension	0.540	0.674	0.594	0.75
Recurrence error	0.540	0.376	0.474	18.13
RankMe (h)	0.328	0.312	0.321	4.00
RankMe (z)	0.493	0.564	0.522	1.46
α -ReQ slope	0.413	0.389	0.404	1.46
LiDAR (z)	0.120	0.130	0.124	85.03

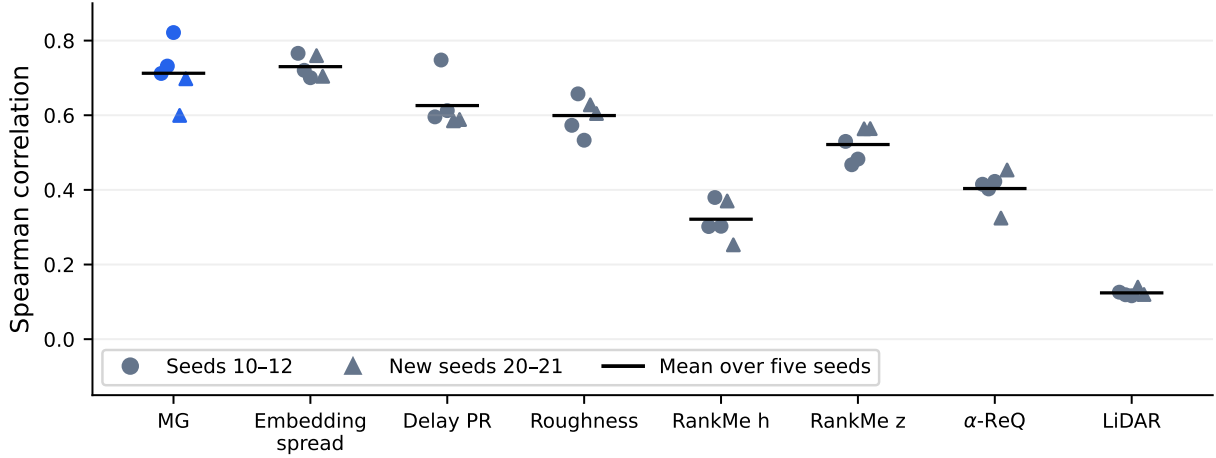


Figure 8: SSL ranking on individual seeds. Circles are the initial test seeds; triangles are the two later seeds. Horizontal bars show five-seed means. MG retains useful ranking on seeds 20–21, while embedding spread has the higher pooled mean.

Table 23: Archived 95% pointwise configuration-bootstrap intervals for mean $\rho_{\text{MG}} - \rho_{\text{comparison}}$. Seed uncertainty and multiple comparisons are not accounted for by these intervals.

Comparison	$\Delta\rho$ lower	$\Delta\rho$ upper
Embedding spread	-0.192	0.190
Spectral entropy	-0.050	0.330
Roughness	-0.099	0.363
Delay covariance PR	-0.073	0.304
Correlation dimension	-0.050	0.326
Recurrence error	0.035	0.446
RankMe (h)	0.078	0.771
RankMe (z)	-0.025	0.461
α -ReQ slope	0.042	0.613
LiDAR (z)	0.180	1.053

Table 24: All frozen SSL selectors over the five evaluated seeds. Score signs and logs were chosen only on development runs. The secondary MG setting uses $\tau = 4, k = 50$.

Statistic	Log	Mean ρ	Regret (pp)
MG	grad_norm	0.712	0.41
out_std	out_std	0.730	1.46
spectral_entropy	grad_norm	0.595	1.00
linear_pr	grad_norm	0.626	1.08
crossings	loss	0.586	1.02
MG_t4k50	loss	0.622	1.60
roughness	loss	0.599	1.20
mean	grad_norm	0.548	3.15
corr_dim	loss	0.594	0.75
self_repeat	grad_norm	0.474	18.13
rel_change	grad_norm	0.539	1.92
lag1	loss	0.565	1.40
rankme_z	rankme_z	0.522	1.46
rel_std	grad_norm	0.482	1.17
twonn	param_norm	0.459	85.03
perm_entropy	probe_h_norm	0.431	85.03
abs_alpha1	abs_alpha1	0.404	1.46
alpha_h	alpha_h	0.404	1.46
det_std	probe_h_norm	0.359	85.03
rankme_h	rankme_h	0.321	4.00
recurrence_rate	loss	0.316	1.21
lidar_z	lidar_z	0.124	85.03

Failure detection, checkpoint selection, and cost. On seeds 10–12, a run is labeled unsuccessful when final probe accuracy falls below the mean random-initialization accuracy for the same seed plus 0.02. This definition measures weak downstream performance; it is not a proof of geometric collapse. Both the final and step-1,500 MG rules achieve AUROC 1.0, as do several other scalar rules. These detection rules select loss, rather than the gradient norm used for ranking. They were not confirmed on seeds 20–21. For checkpoint selection, spectral entropy has regret 0.00174, versus approximately 0.0069 for MG and 0.0172 for choosing the last checkpoint; this is another task with a separately fitted rule.

The saved CPU measurements average 30.83 ms for MG on a 1,000-sample window, versus 0.070 ms for roughness and 0.161 ms for spectral entropy. Gradient-norm logging needs no extra forward or backward pass, but its reduction is $O(P)$ and is not free. The archived RankMe time includes both h and z SVDs, and the original MG timing averages several logs under concurrent workers; these totals are not a fair single-score speed comparison. MG selection uses the saved log; embedding proxies require model access and image evaluations. Large-model scaling was not measured.

I.2 Warning of slingshot loss spikes

Model, event, and decision. We train a pre-layer-normalized transformer with two layers, width 64 and four heads on addition modulo 97, using Adam with zero weight decay for 8,000 steps. Inputs are two operands and a delimiter; the answer is predicted at the last position. Four configurations cross selected learning rates (10^{-3} or $2 \cdot 10^{-3}$), second-moment coefficients (0.98 or 0.95), and training fractions (0.3 or 0.5). Batch size is 256 and warm-up lasts 100 steps. This produces slingshot events (Thilak et al., 2022); a preliminary character-language-model pilot did not yield the desired events and is not the benchmark reported here.

An event begins when the next ten-step mean of $\log_{10}$ loss exceeds its preceding 200-step median by more than one and by more than six MAD-based standard deviations. Events have a 300-step exclusion interval. This use of future samples defines ground truth only. Monitors read completed 500-step windows every 50 steps, using loss or gradient/update/parameter norms; the first three are log-transformed. MG uses $E = 20, \tau = 1, k = 20, T = 19$. A successful warning occurs within 500 steps before onset. Windows ending at or within 500 steps after an onset are excluded from scoring; nearby false positives are merged within 500 steps. The false-positive rate is per 10,000 scored steps, not the entire training duration.

We choose the log, sign, threshold, and level/change rule using seeds 0–3, targeting at most two false positives per

10,000 steps, then evaluate seeds 10–13. The 16 test runs contain 23 scored events. The held-out false-positive rate need not meet the development constraint. Table 25 includes all primary method comparisons.

Table 25: Spike warnings on the held-out transformer runs. Hits are events warned within 500 steps; FP/10k counts merged false warnings per scored training steps. AUROC uses the frozen score and future-onset label at evaluation times.

Score	Hits	FP/10k	AUROC
MG	22/23	2.44	0.949
MG_t4k50	20/23	2.66	0.927
self_repeat	23/23	2.76	0.960
spectral_entropy	21/23	2.02	0.962
roughness	23/23	2.44	0.960
perm_entropy	19/23	2.23	0.881
recurrence_rate	20/23	2.44	0.726
corr_dim	12/23	2.44	0.882
twonn	21/23	2.12	0.889
linear_pr	23/23	2.66	0.959
crossings	17/23	2.44	0.841
lag1	23/23	2.34	0.951
det_std	23/23	2.76	0.931
var	22/23	2.44	0.923
loss threshold	23/23	2.34	0.946
grad-norm threshold	22/23	2.34	0.917
update-norm level	15/23	2.34	0.788
param-norm level	14/23	3.40	0.716
INT attn-logit max	4/23	1.70	0.645
INT attn entropy	15/23	1.59	0.911
INT output logZ	20/23	1.91	0.827

MG warns of 22/23 events with median lead 469.5 steps and precision 0.456. The long lead relative to the scoring horizon indicates an extended warning regime. Roughness warns of all events at the same false-positive rate for less computation. Attention scores are weaker for this slingshot mechanism; other instability mechanisms remain untested. Both the event definition and MG input use loss, so this is event forecasting rather than independent evidence of dimensional simplification. No intervention or training-quality improvement was tested.

I.3 Reset rules, learning-rate schedules and transfer

Plasticity and resets. Permuted-MNIST MLPs learn 40 successive tasks, 400 updates per task, with homogeneous or varying-noise input families. Rules select full-network resets at task boundaries. Development uses seeds 0–3, initial evaluation uses 10–14, and actual reset continuations use 20–21 crossed with six conditions, yielding 12 streams per family. MG improves online accuracy relative to never resetting, but dormant-neuron and weight-norm rules are better (Table 21). The 12 streams are not 12 independent initialization seeds.

Learning-rate decisions. A small CIFAR-10 CNN is trained under ten conditions varying batch size, initial step size, label noise, data volume, and budget. Development uses 12 runs and evaluation 24 runs. Each rule observes a constant-rate training path; saved checkpoints supply measured continuations with a tenfold rate reduction. MG reaches 0.574 final accuracy, improving on the constant-rate baseline, while the best fixed late reduction reaches 0.590. A second stop-after-reduction test gives only a 0.09 percentage-point advantage over a fixed schedule at matched average compute, with interval $[-0.34, 0.45]$ percentage points. Stationarity of a log is therefore not sufficient to determine the best time to reduce the learning rate.

Games and detector transfer. The game study constructs weakly coupled matching-pennies cycles and predicts a cycle category as a proxy for choosing a population size. It does not run a downstream PSRO improvement experiment. MG classifies 52.3% of 216 new games correctly; full-state PCA is nearly exact and a scalar harmonic count reaches 71.3%. In a separate transfer study, a detector fitted on one MNIST MLP is applied across width, objective, precision, restart, and dataset changes. MG detects 26/87 events with 10/58 false positives, versus 85/87 and 9/58 for detrended spread. The original threshold does not transfer reliably across these settings.